5.1 “移近—归约”分析法

5.2 算符优先分析法

5.3 LR分析鸟瞰

5.4 LR (0)分析

5.5 SLR (1)分析

5.6 LR (1)分析

5.7 LALR (1)分析NO

5.8 LR分析对二义文法的应用 NO

5.9 LR分析的错误处理与恢复NO

5.10 语法分析器的自动生成与YACC NO

5.7.1　LALR(1)分析实现思想

5.7.2　LALR(1)分析表的构造

分析功能： LR(1) ⊃ SLR(1) ⊃ LR(0)

分析器开销： LR(1) ↑ SLR(1) LR(0) ↓

存储开销 ⇨ LR(0)、SLR(1)

分析功能 ⇨ LR(1)

⇨ LALR(1)

LALR(1)

→ **Look −Ahead LR(1)**

LALR(1)分析表状态数 = LR(0)的状态数

LALR(1)分析功能 ⊆ LR(1)

例5.17 设有文法$G(S')$为

(1) $S' \rightarrow S$

(2) $S \rightarrow L=R$

(3) $S \rightarrow R$

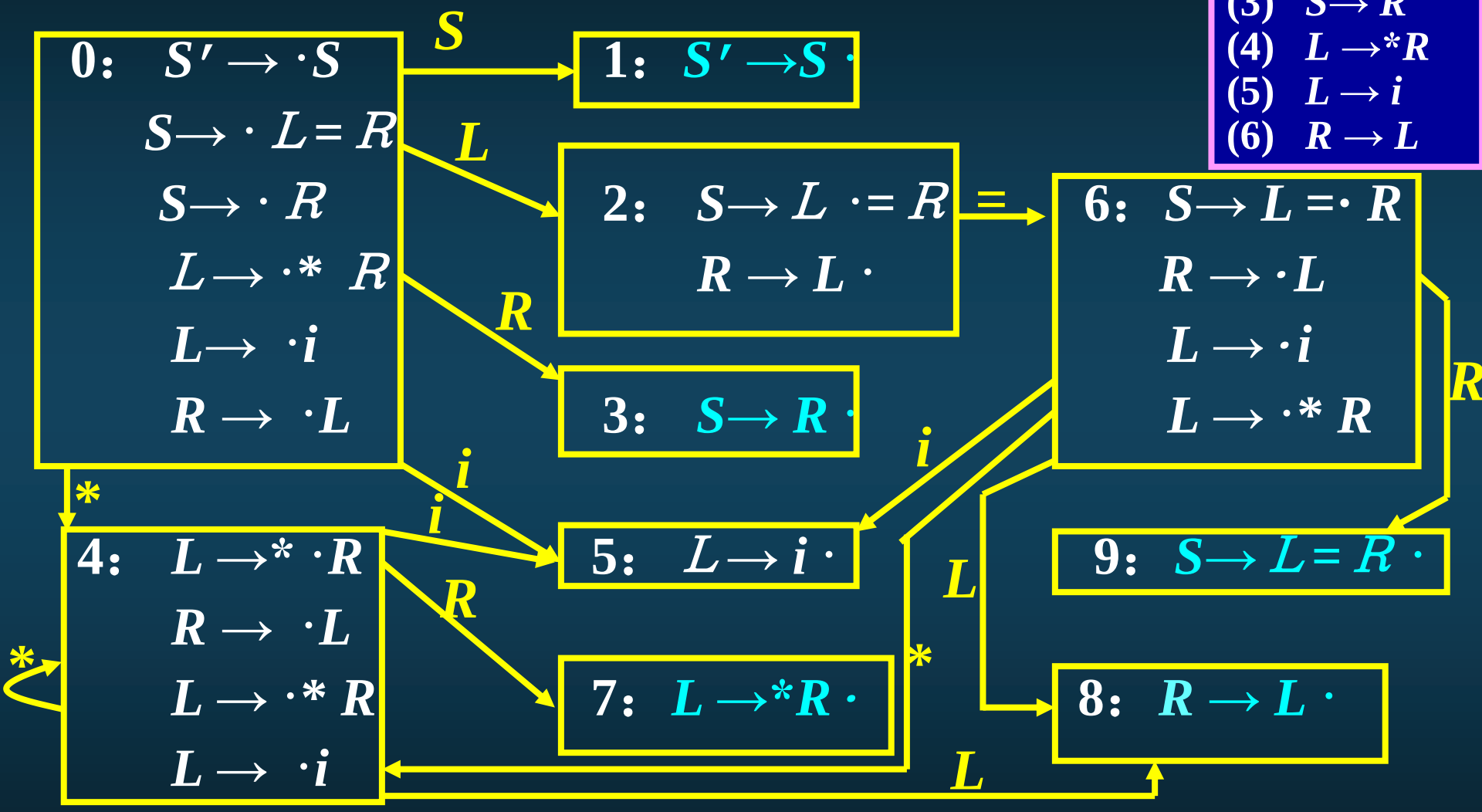
(4) $L \rightarrow *R$

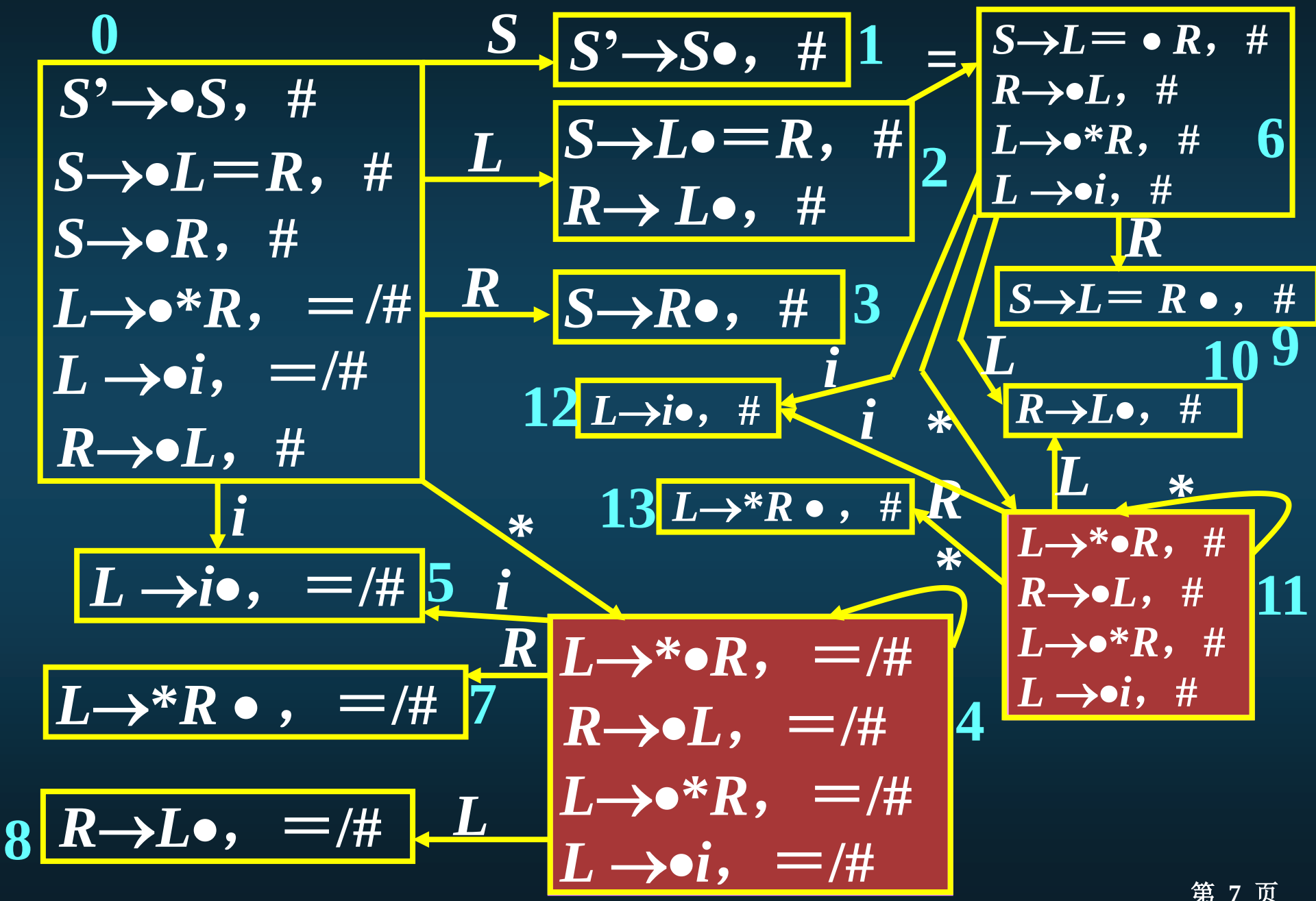
(5) $L \rightarrow i$

(6) $R \rightarrow L$

(1) $S' \to S$
(2) $S \to L=R$
(3) $S \to R$
(4) $L \to *R$
(5) $L \to i$
(6) $R \to L$

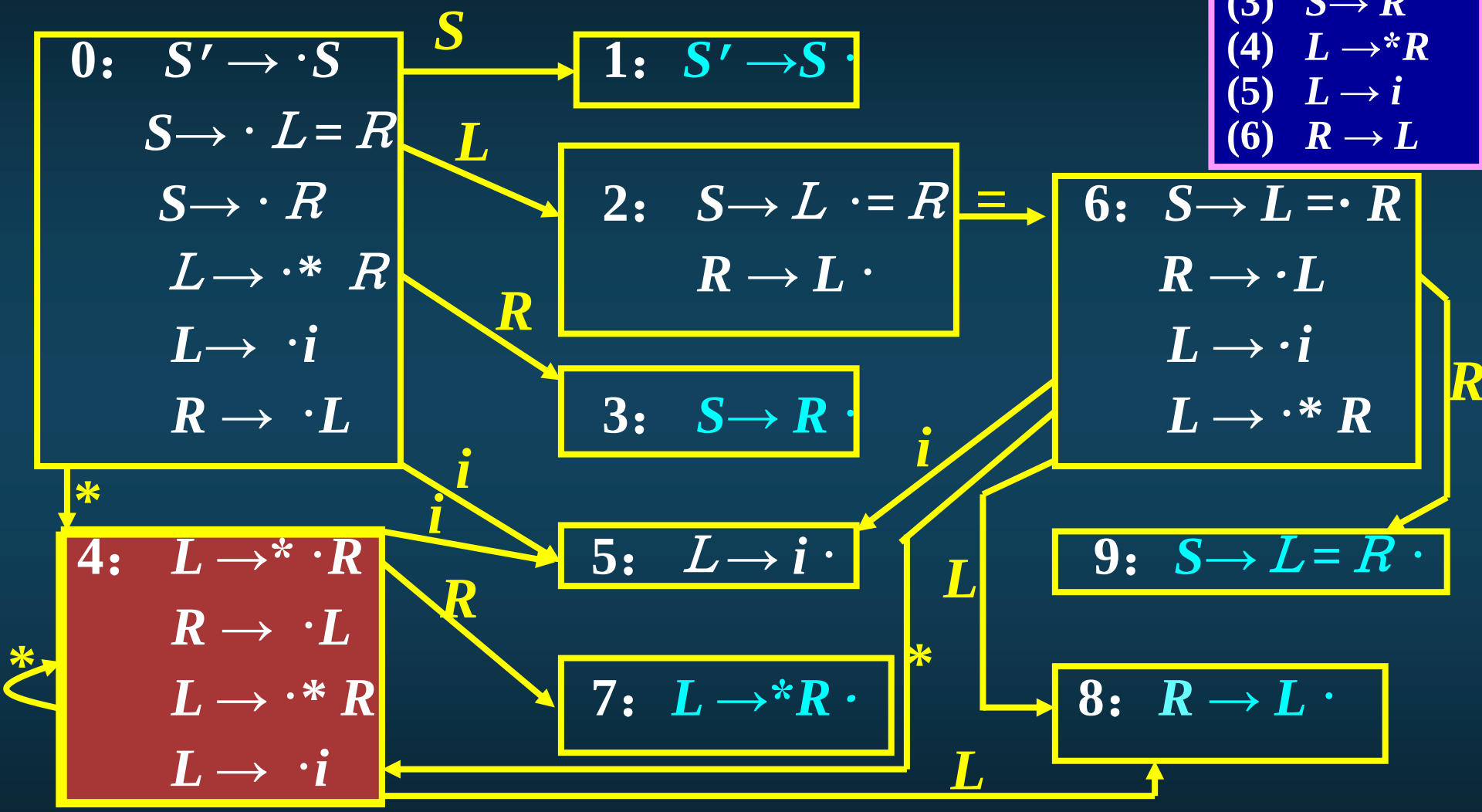
$G(S')$的识别关于LR(0)项目的活前缀的DFA：

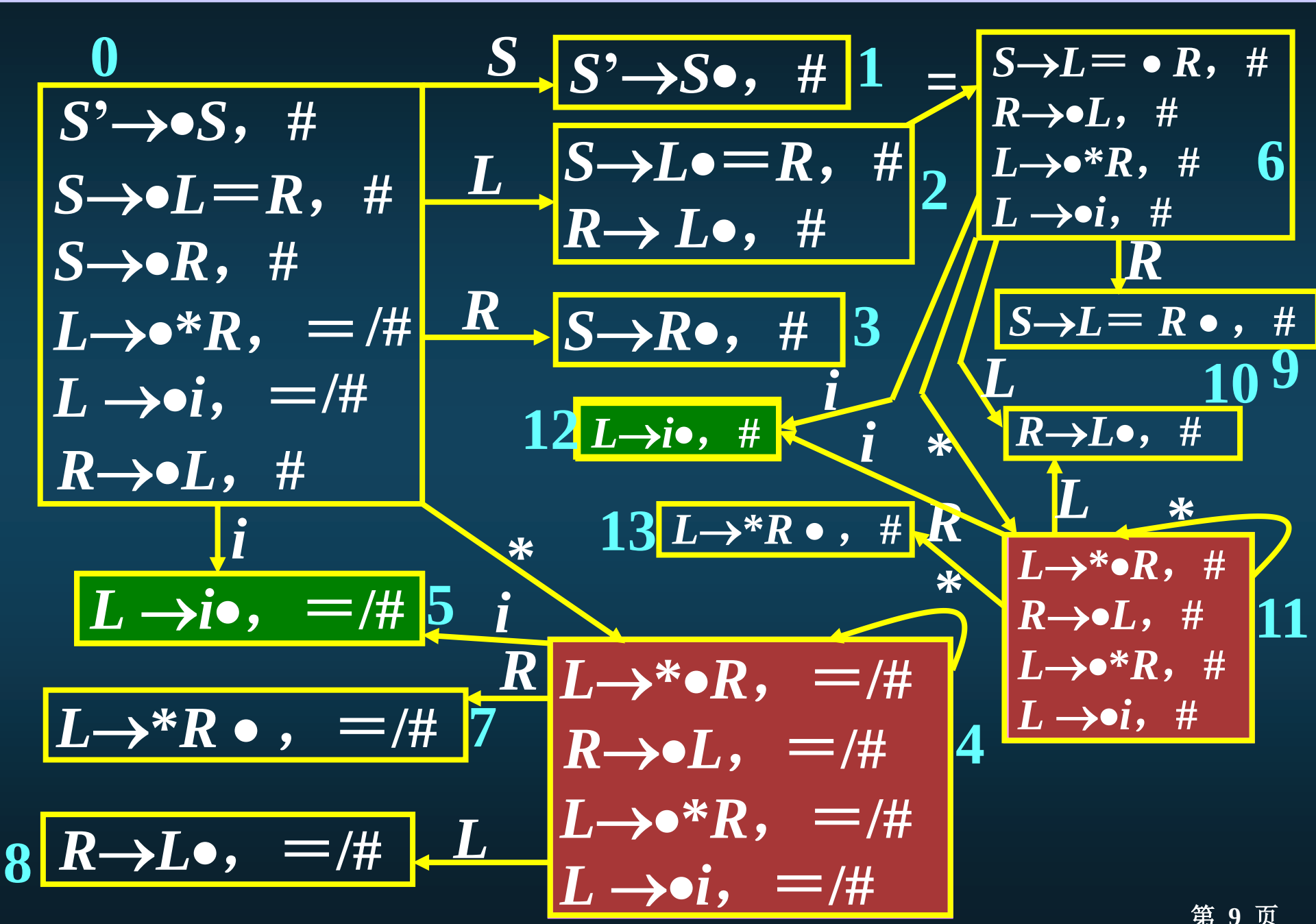



0
S'→•S，#
S→•L=R，#
S→•R，#
L→•*R，=/#
L→•i，=/#
R→•L，#
S
S'→S•，#
1
L
S→L•=R，#
R→L•，#
2
R
S→R•，#
3
=
S→L=•R，#
R→•L，#
L→•*R，#
L→•i，#
6
R
S→L=R•，#
9
L
R→L•，#
10
i
L→i•，#
12
i
*
L
*
L→*•R，#
R→•L，#
L→•*R，#
L→•i，#
11
R
L→*R•，#
13
i
L→i•，=/#
5
*
i
R
L→*•R，=/#
R→•L，=/#
L→•*R，=/#
L→•i，=/#
4
*
L→*R•，=/#
7
L
R→L•，=/#
8

G(S')的识别关于LR(0)项目的活前缀的DFA：

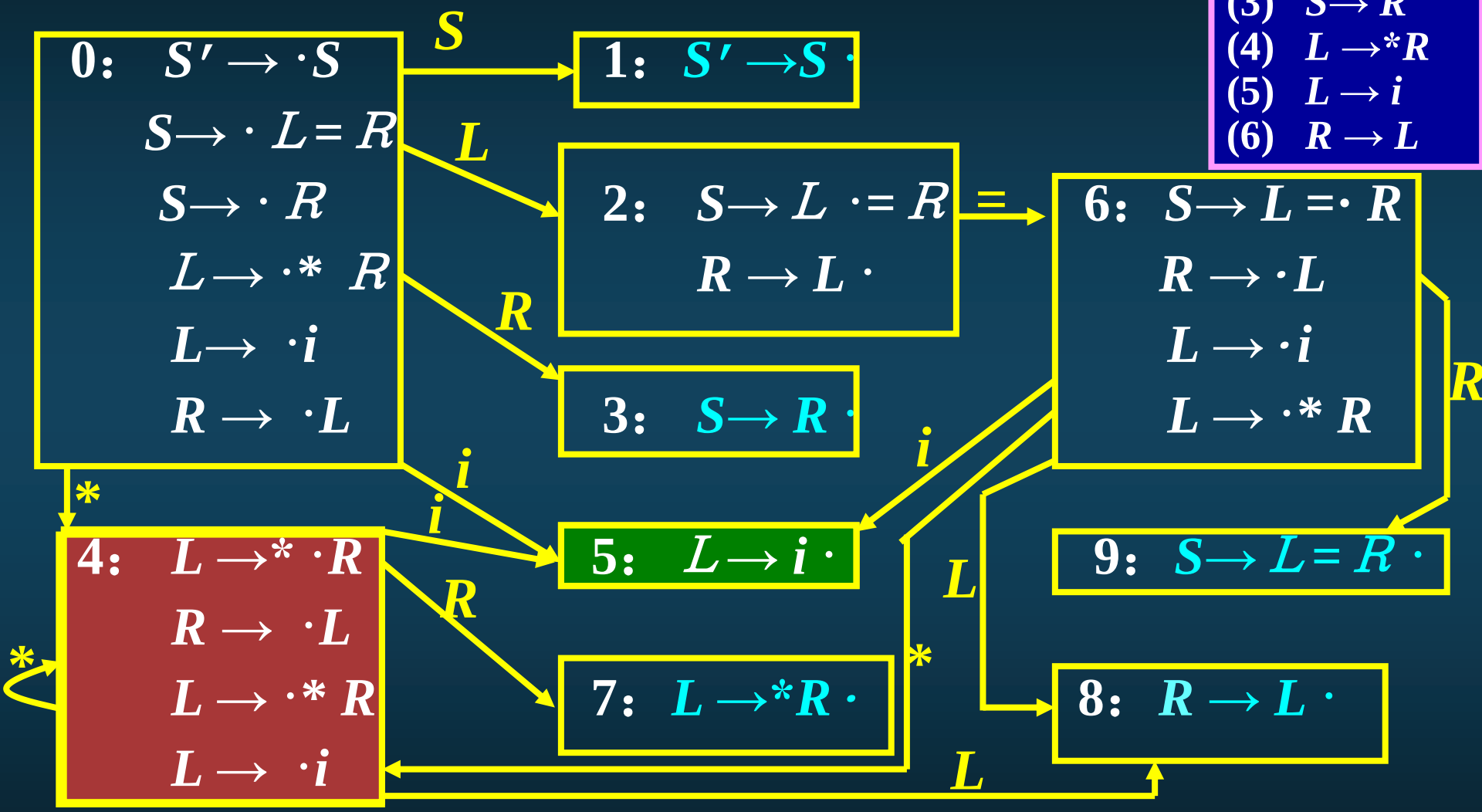
(1) $S' \to S$
(2) $S \to L=R$
(3) $S \to R$
(4) $L \to *R$
(5) $L \to i$
(6) $R \to L$

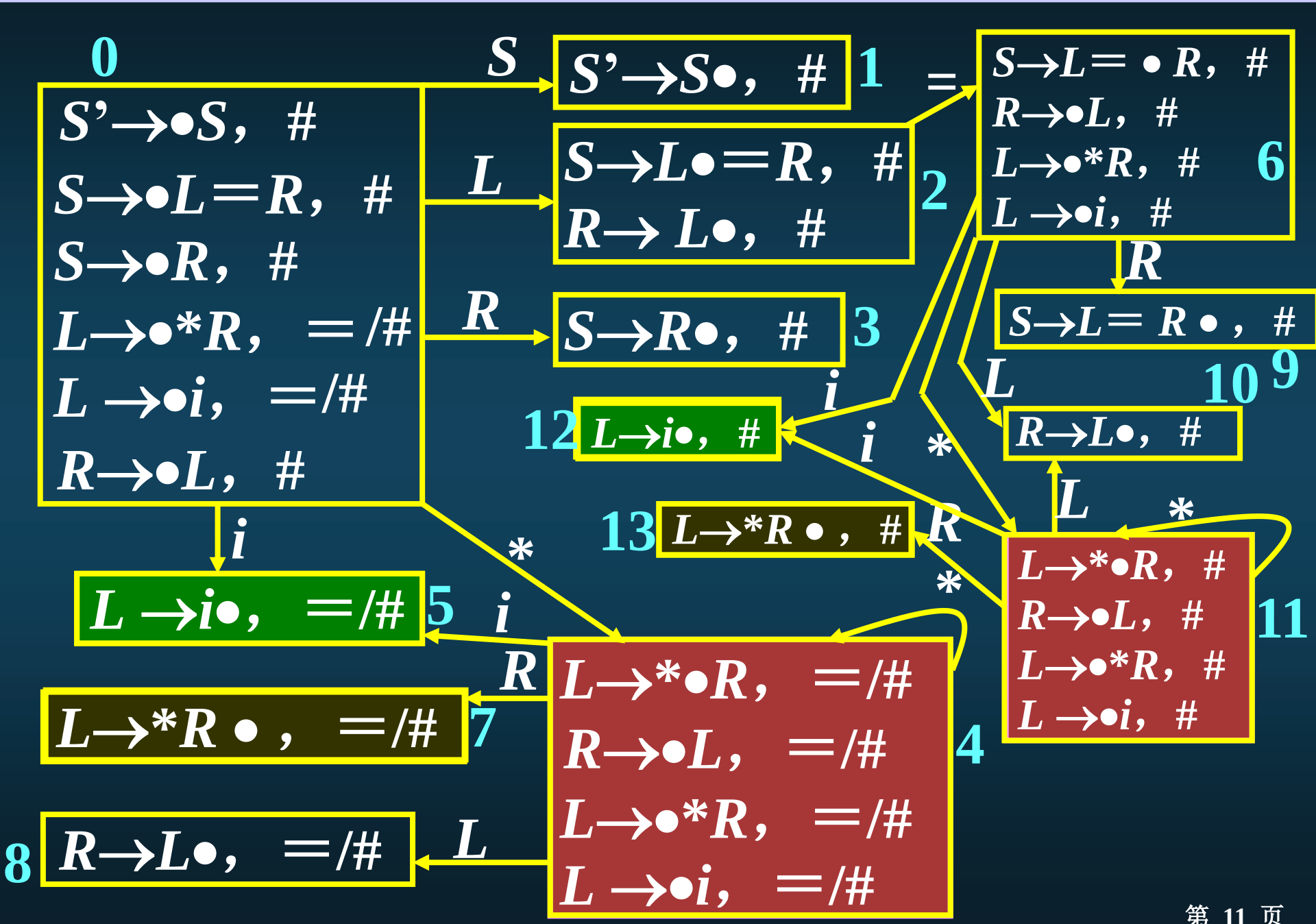



0
S'→•S，#
S→•L=R，#
S→•R，#
L→•*R，=/#
L→•i，=/#
R→•L，#
S
S'→S•，#
1
L
S→L•=R，#
R→L•，#
2
R
S→R•，#
3
=
S→L=•R，#
R→•L，#
L→•*R，#
L→•i，#
6
R
S→L=R•，#
9
L
R→L•，#
10
i
L→i•，#
12
i
*
L
*
L→*•R，#
R→•L，#
L→•*R，#
L→•i，#
11
R
L→*R•，#
13
i
L→i•，=/#
5
*
i
R
L→*•R，=/#
R→•L，=/#
L→•*R，=/#
L→•i，=/#
4
*
L→*R•，=/#
7
L
R→L•，=/#
8

G(S')的关于LR(0)项目的识别活前缀的DFA：

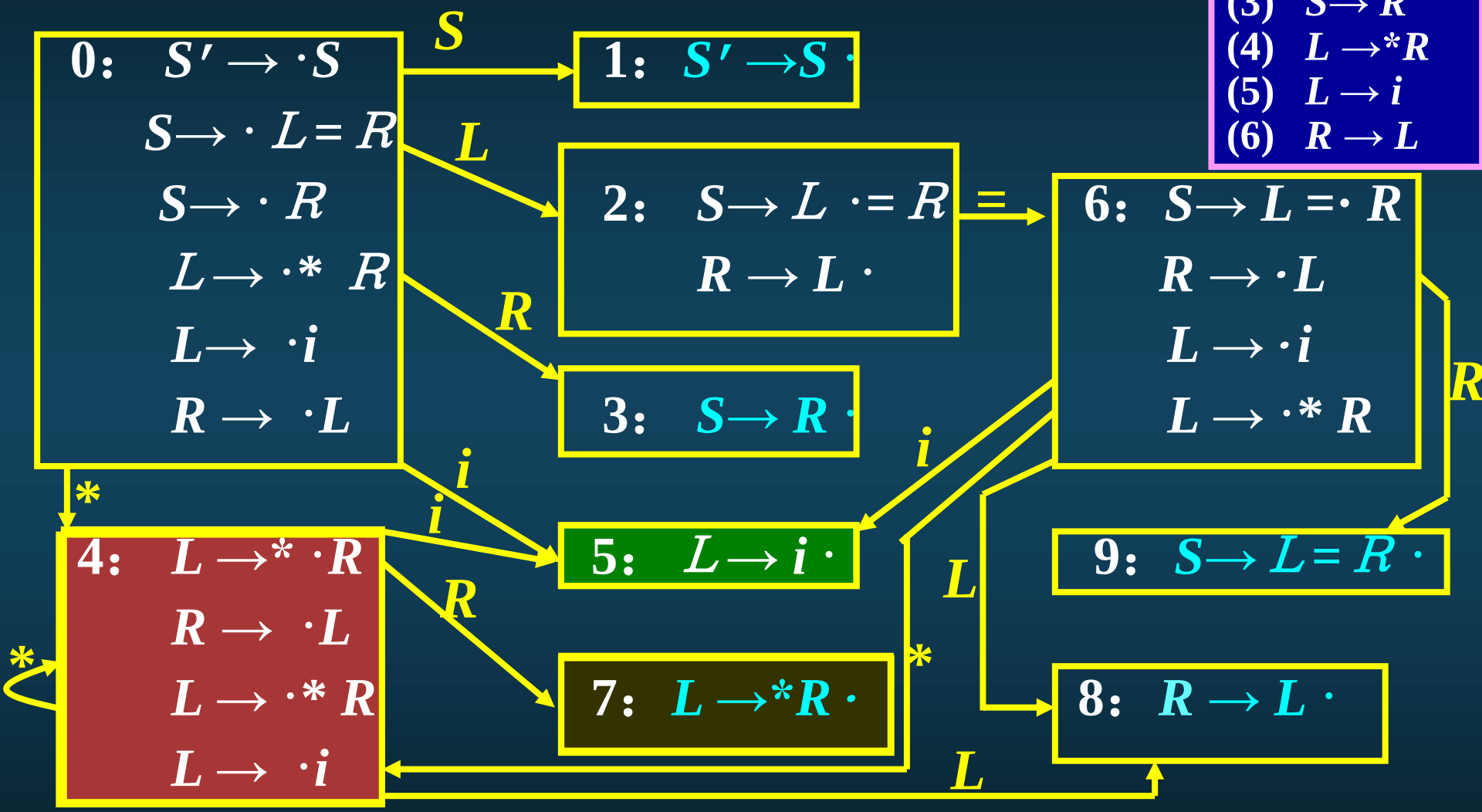
(1) S'→S
(2) S→ L=R
(3) S→ R
(4) L →*R
(5) L → i
(6) R → L

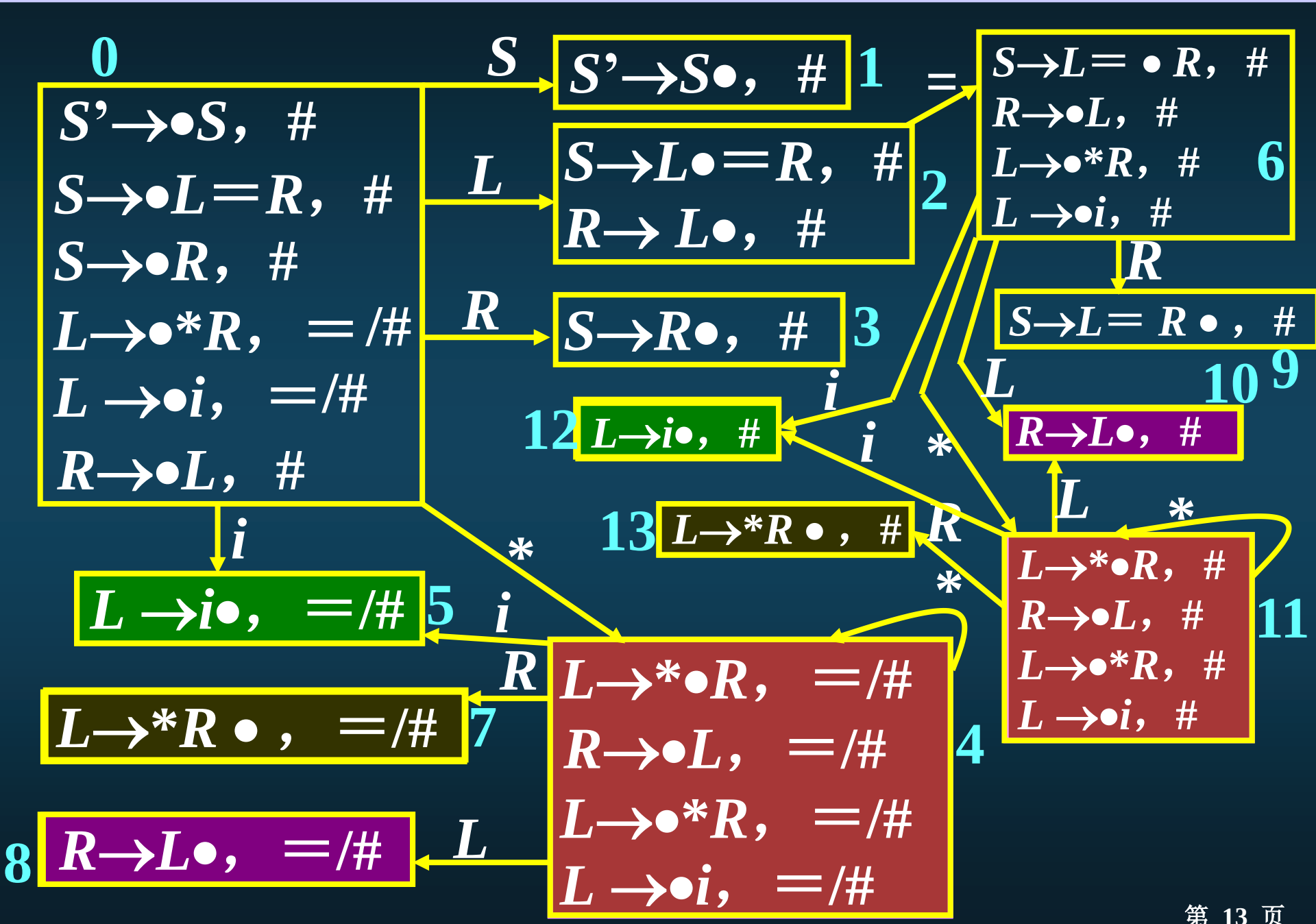



0
S'→•S，#
S→•L=R，#
S→•R，#
L→•*R，=/#
L→•i，=/#
R→•L，#
S
S'→S•，#
1
L
S→L•=R，#
R→L•，#
2
R
S→R•，#
3
=
S→L=•R，#
R→•L，#
L→•*R，#
L→•i，#
6
R
S→L=R•，#
9
L
R→L•，#
10
i
L→i•，#
12
i
*
L
*
L→*R•，#
13
R
*
L→*•R，#
R→•L，#
L→•*R，#
L→•i，#
11
i
L→i•，=/#
5
*
i
R
L→*•R，=/#
R→•L，=/#
L→•*R，=/#
L→•i，=/#
4
L→*R•，=/#
7
L
R→L•，=/#
8

G(S')的关于LR(0)项目的识别活前缀的DFA：

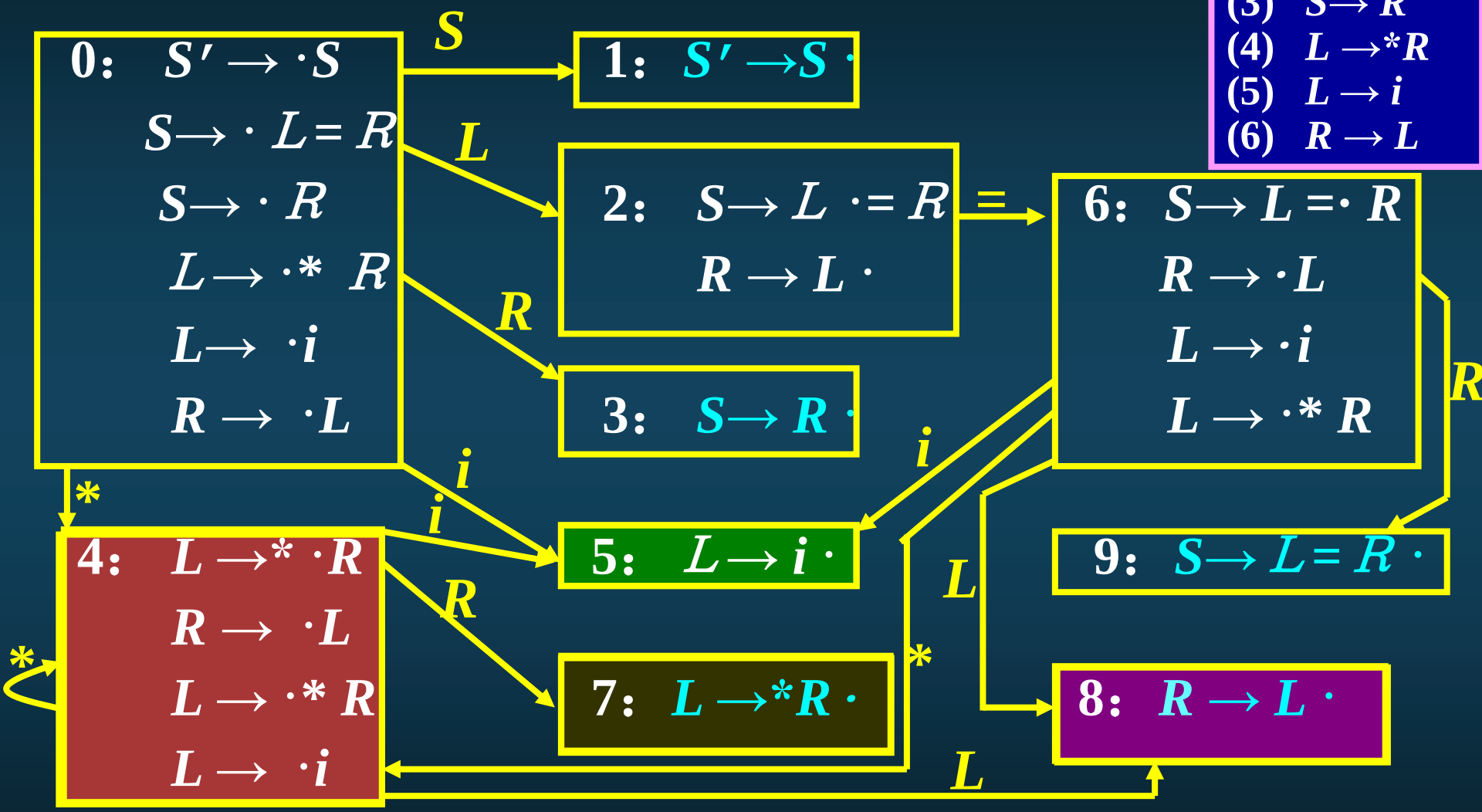
(1) S'→S
(2) S→ L=R
(3) S→ R
(4) L →*R
(5) L → i
(6) R → L

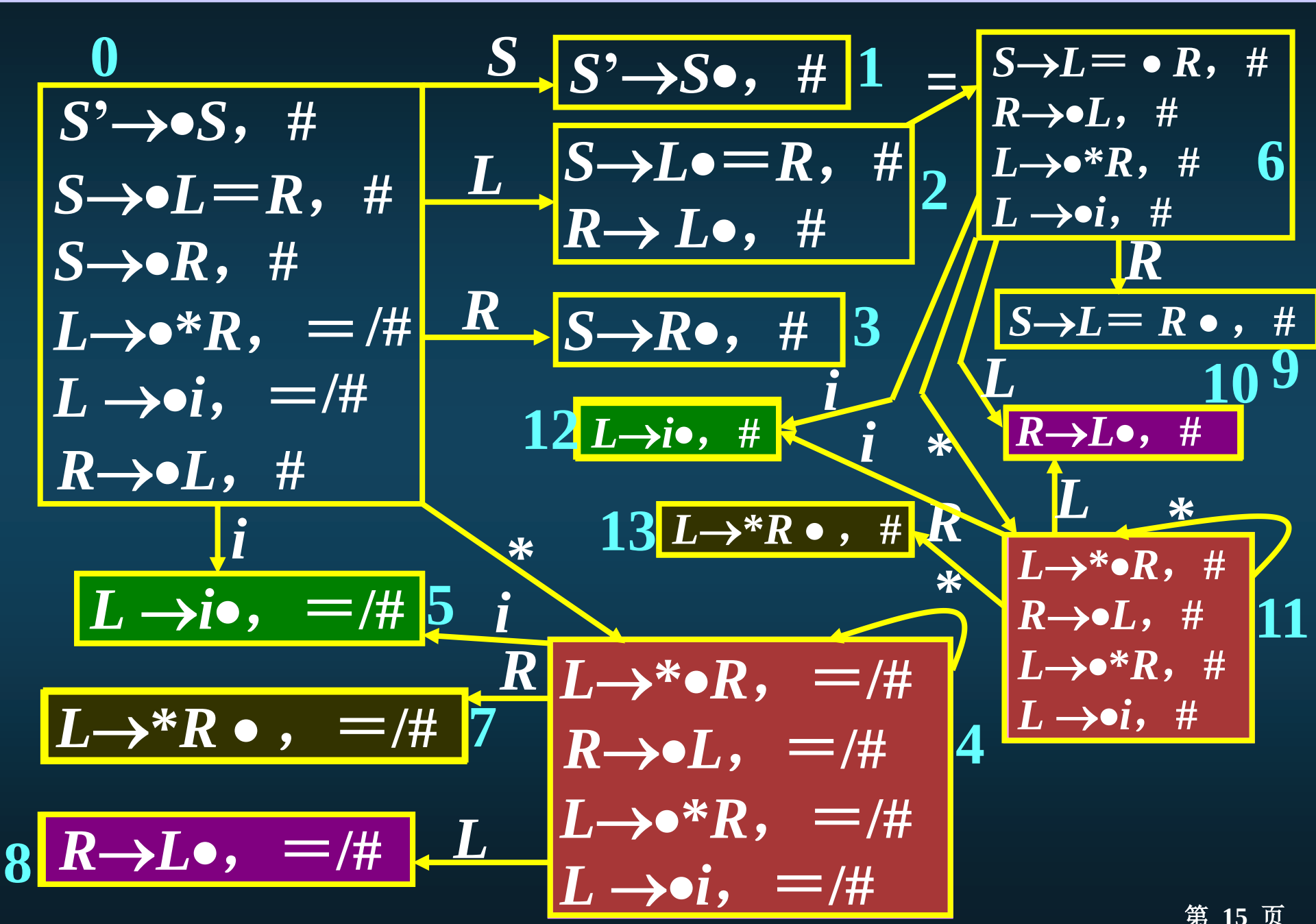



0
S'→•S，#
S→•L=R，#
S→•R，#
L→•*R，=/#
L→•i，=/#
R→•L，#
S
S'→S•，#
1
L
S→L•=R，#
R→L•，#
2
R
S→R•，#
3
=
S→L=•R，#
R→•L，#
L→•*R，#
L→•i，#
6
R
S→L=R•，#
9
L
10
R→L•，#
i
L→i•，#
12
i
*
L
*
L→*R•，#
13
R
*
L→*•R，#
R→•L，#
L→•*R，#
L→•i，#
11
i
L→i•，=/#
5
*
i
R
L→*•R，=/#
R→•L，=/#
L→•*R，=/#
L→•i，=/#
4
L→*R•，=/#
7
L
8
R→L•，=/#

G(S')的关于LR(0)项目的识别活前缀的DFA：

(1) S'→S
(2) S→ L=R
(3) S→ R
(4) L →*R
(5) L → i
(6) R → L




0
S'→•S，#
S→•L=R，#
S→•R，#
L→•*R，=/#
L→•i，=/#
R→•L，#
S
S'→S•，#
1
L
S→L•=R，#
R→L•，#
2
R
S→R•，#
3
=
S→L=•R，#
R→•L，#
L→•*R，#
L→•i，#
6
R
S→L=R•，#
9
L
10
R→L•，#
i
L→i•，#
12
i
*
L
*
L→*R•，#
13
R
*
L→*•R，#
R→•L，#
L→•*R，#
L→•i，#
11
i
L→i•，=/#
5
*
i
R
L→*•R，=/#
R→•L，=/#
L→•*R，=/#
L→•i，=/#
4
L→*R•，=/#
7
L
8
R→L•，=/#

对比文法*G*(*S′*)的LR(1)的*C*和LR(0)的*C*

LR(0)的*C*		LR(1)的*C*
4	分裂为	4、11
5	分裂为	5、12
7	分裂为	7、13
8	分裂为	8、10

每个项目集仅搜索符不同，LR(0)项目相同

定义5.16 （同心项目集）

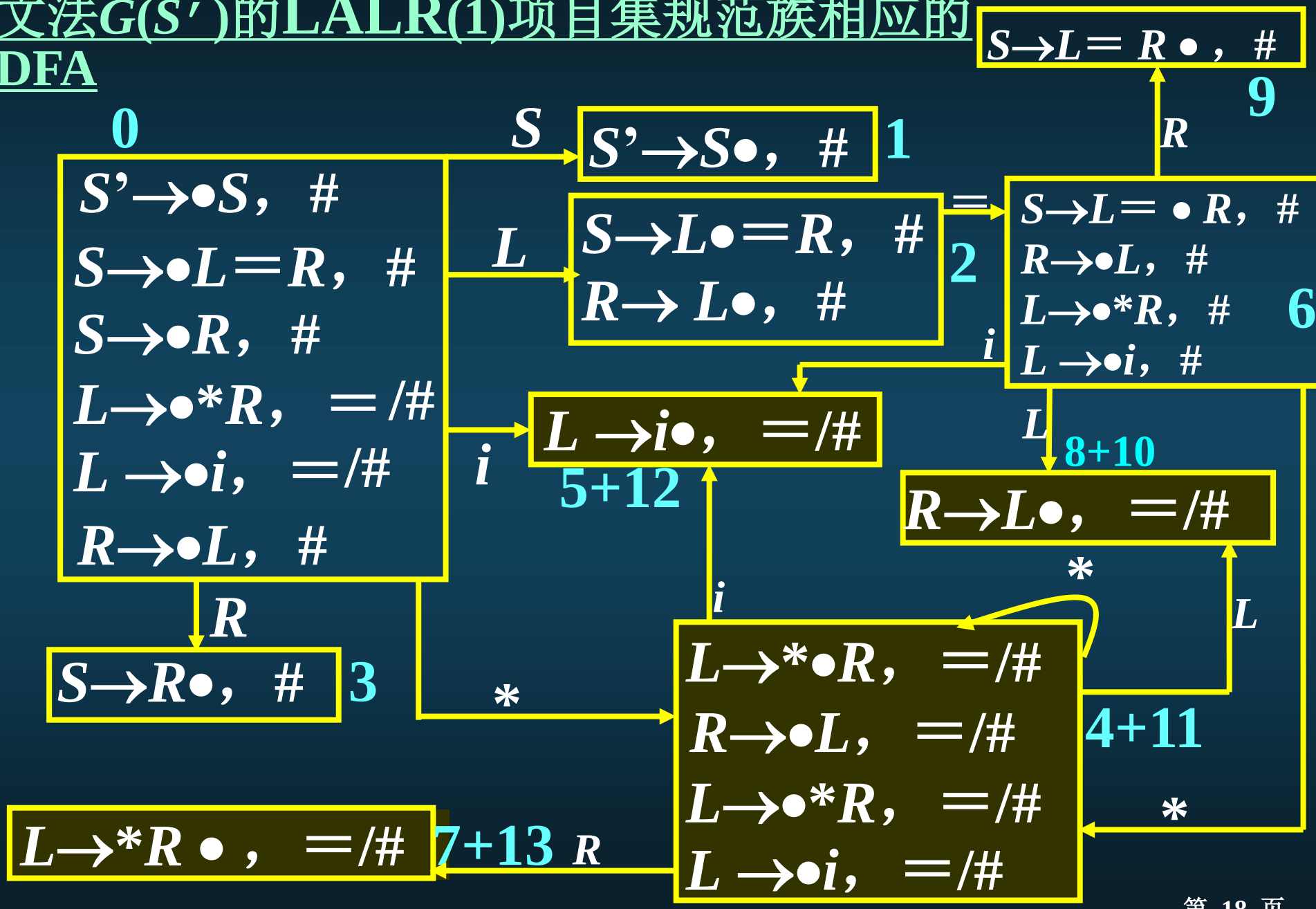
对文法G的 LR(1) 项目集规范族，若存在两个(或两个以上)项目集I_0、I_1，其中I_0、I_1项目集中的LR(0)项目相同，仅搜索符不同，则称I_0 、I_1为G的LR(1)的同心项目集。或称I_0 、I_1具有相同的心。

合并LR(1)的C中的同心项目集=> LALR (1)的C

缩小DFA分析的状态数

DFA的状态转移或GO函数做相应的修改。

文法G(S′)的LALR(1)项目集规范族相应的DFA



注意：

1. LR(1)的*C*无冲突 ⇒ LALR(1)的*C*

LALR(1)的*C* { 无冲突：构造LALR(1)分析表
有冲突：构造LR(1)的分析表

2. LALR(1)的*C*无冲突成立，分析能力同LR(1)。

确定串中错误前有时会比LR(1)多产生若干步归约。

5.7.1　LALR(1)分析实现思想

5.7.2　LALR(1)分析表的构造

- **算法5.11 （ LALR(1)分析表构造）**

输入： 拓广的文法G'及文法G'的LALR(1)项目集规范族C和GO函数

输出： 文法G' 的LALR(1)分析表

方法： 设$C=\{I_0, I_1, \ldots, I_n\}$,分析表的状态$=\{0, 1, \ldots, n\}$。

①若GO $(I_k, a)=I_j$，$a \in V_T$， 则置action $(K, a)=S_j$;

若GO $(I_k, A)=I_j$， $A \in V_N$，则置GOTO$[K, A]=j$。

②若［$A\rightarrow \alpha \cdot, a$］$\in I_k$，则置action$(K, a)=r_j$，其中

$A\rightarrow \alpha$为文法G的第j个产生式;

③ 若接受项目［$S' \rightarrow S \cdot$， #］$\in I_k$，则置action $(K, \#)=$acc;

④分析表中不能用①至③规则填入信息的元素，则置“出错标志”

- **算法5.11　（ LALR(1)分析表构造）**

输入： 拓广的文法*G'*及文法*G'*的识别LALR(1)项目有效活前缀的DFA

输出： 文法*G'* 的LALR(1)分析表

方法： 设状态集$Q=\{0, 1, \ldots, n\}$＝分析表的状态

① 若$f(k, a)=j$，$a \in V_T$，则置 action $(k, a)=S_j$;

若$f(k, A)=j$，$A \in V_N$，则置GOTO$[K, A]=j$。

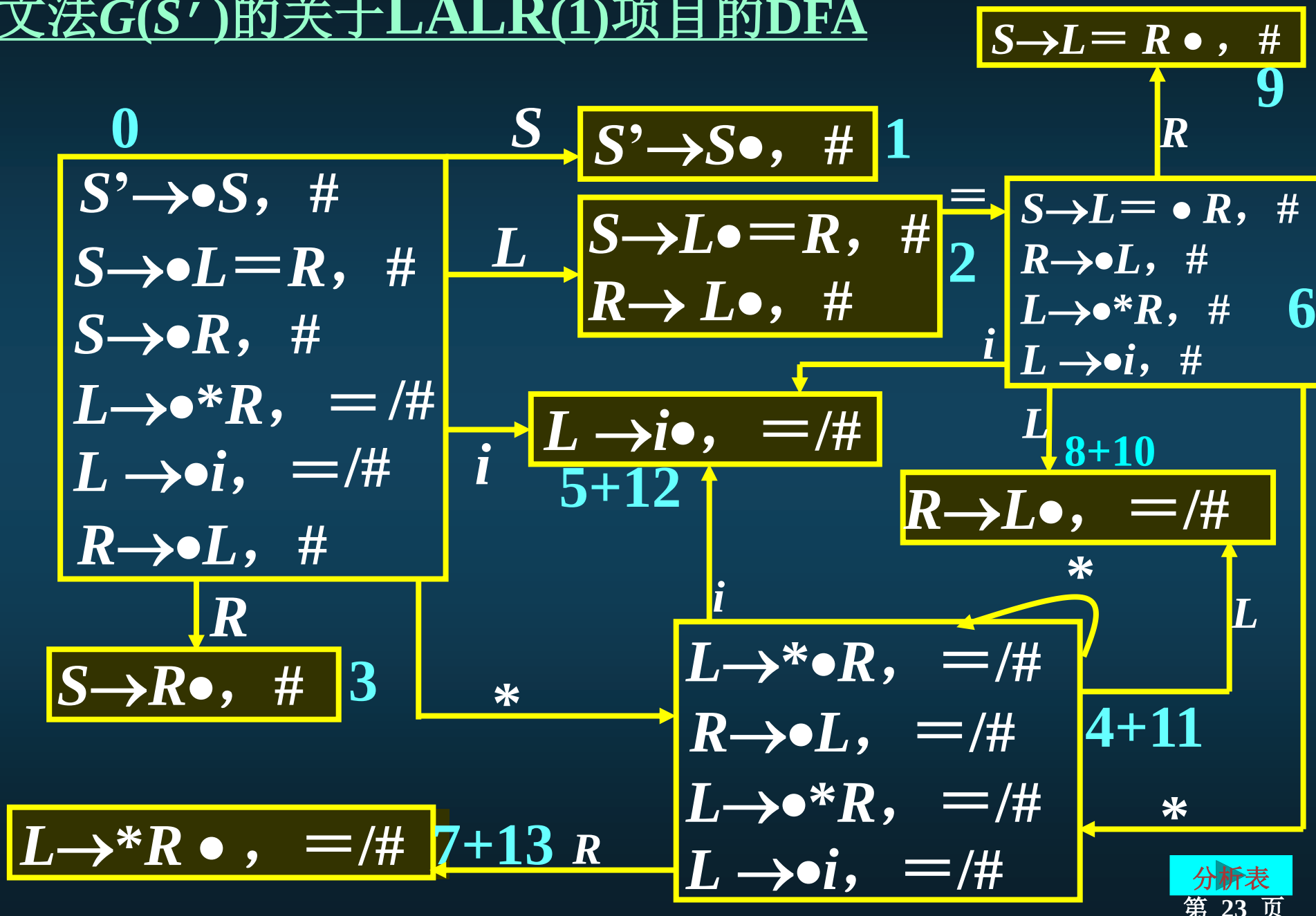
②若［$A\rightarrow \alpha \cdot, a$］$\in k$，则置action$(K, a)=r_j$，其中

$A\rightarrow \alpha$为文法*G*的第*j*个产生式;

③ 若接受项目［$S' \rightarrow S\cdot$，＃］$\in k$，则置action $(K, \#)=$acc;

④分析表中不能用①至③规则填入信息的元素，则置“出错标志”

文法G(S′)的关于LALR(1)项目的DFA



分析表

文法G(S′)LALR(1)分析表

(1) $S' \to S$
(2) $S \to L=R$
(3) $S \to R$
(4) $L \to *R$
(5) $L \to i$
(6) $R \to L$

state	Action				Goto		
	=	*	*i*	#	*L*	*R*	*S*
0		S_{4+11}	S_{5+12}		2	3	1
1				acc			
2	S_6			r_6			
3				r_3			
4+11		S_{4+11}	S_{5+12}		8+10	7＋13	
5+12	r_5			r_5			
6		S_{4+11}	S_{5+12}		8+10	9	
7+13	r_4			r_4			
8+10	r_6			r_6			
9				r_2			

DFA

定义5.16

按照LALR(1)的项目集规范族构造的文法*G*的LALR (1)分析表，如果每个入口不含多重定义，则称它为*G*的LALR(1)分析表。具有LALR(1)分析表的文法*G*称为LALR(1)文法。使用 LALR(1) 分析表的 语 法分析器称作LALR(1)分析器。

综述：

1. 构造文法G的LR(1)的项目集规范族C

$$C = \{ I_0 , I_1 , I_2 , \dots , I_n \}$$

2. 合并C中的所有同心的LR(1) 项目集，用M表示合并后的新项目集

$$M = \{ M_0 , M_1 , M_2 , \dots , M_m \}$$ 其中：$m<n$

M称为LALR(1)的项目集规范族。

3. M若没有冲突，据M构造文法G的LALR(1)的分析表。

LALR(1)的项目集规范族若存在冲突，只能是归约— 归约冲突。 (尽管原来的LR(1)的C不冲突)

证明：

设原LR(1)的项目集规范族C中有如下项目集

I_k: $[A \to \alpha \bullet, W_1]$ I_j: $[A \to \alpha \bullet, W_2]$

$[B \to \beta \bullet a\gamma, b]$ $[B \to \beta \bullet a\gamma, c]$

由于I_k与I_j均无冲突，故有

$W_1 \cap \{a\} = \varnothing$ $W_2 \cap \{a\} = \varnothing$

从而 $(W_1 \cup W_2) \cap \{a\} = \varnothing$

现将I_k与I_j合并，有

$I_{k/j}$：

$$[A \rightarrow \alpha \bullet\ ,\ W_1 \cup W_2]$$

$$[B \rightarrow \beta \bullet a\ \gamma,\ \{b\} \cup \{c\}]$$

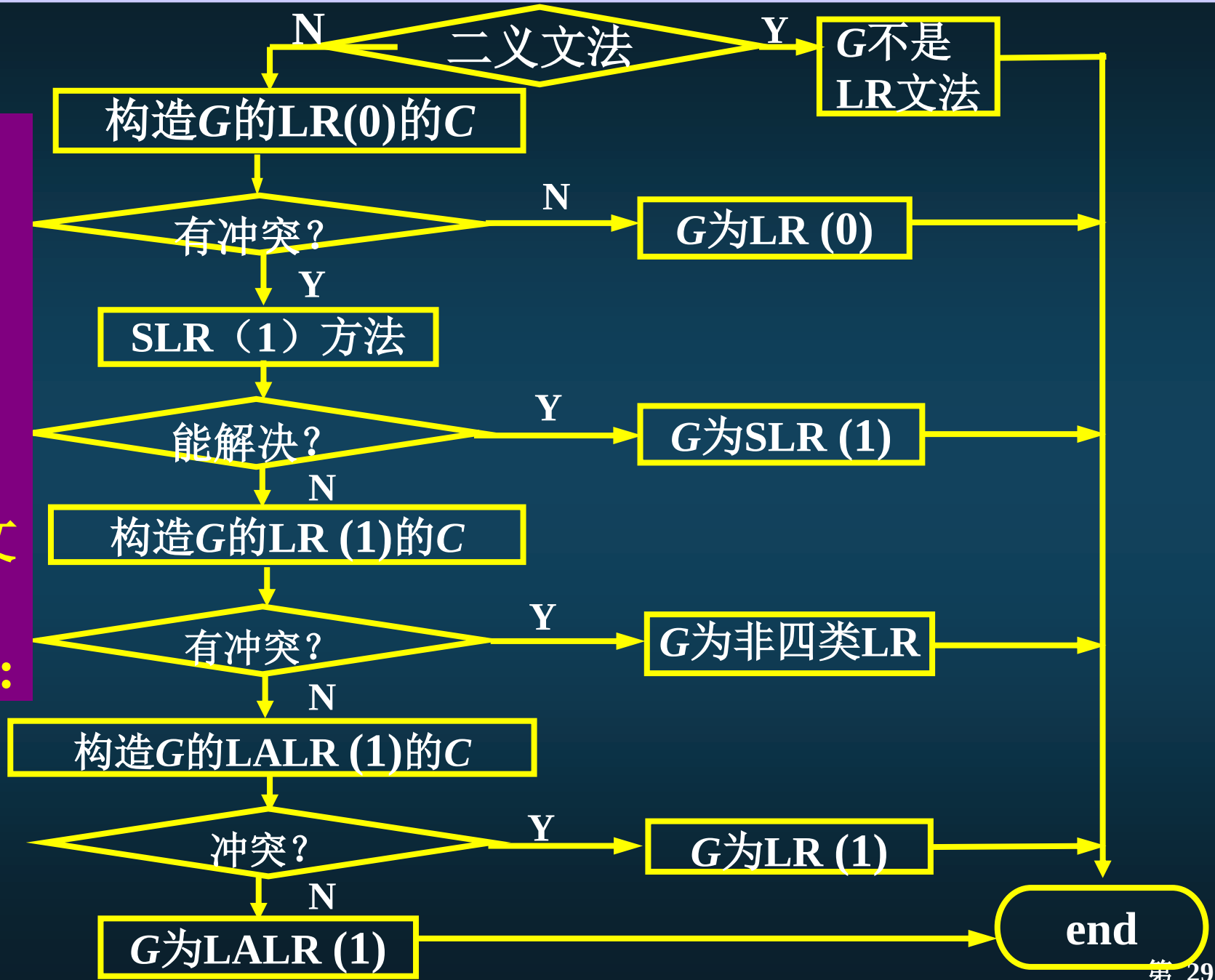
若此时 $I_{k/j}$ 有“移进 — 归约 ”冲突，则必有

$$(W_1 \cup W_2) \cap \{a\} \neq \varnothing$$

与I_k和I_j无冲突的假设矛盾。

∴ 合并同心集后不会引入新的“移进—归约”冲突。 证毕。

判断一个文法属于那一类LR文法的步骤:



例5.18 设有下列文法

$$S \rightarrow Aa \mid bAc \mid Bc \mid bBa$$

$$A \rightarrow d$$

$$B \rightarrow d$$

判断该文法是哪类LR文法？

没有发现有二义性，所以从LR(0)开始判断。

拓广文法：

增加一产生式：$S' \rightarrow S$

0

$S' \to \bullet S$

$S \to \bullet Aa \mid \bullet bAc \mid \bullet Bc \mid \bullet bBa$

$A \to \bullet d$

$B \to \bullet d$

$\xrightarrow{d}$

1

$A \to d \bullet$

$B \to d \bullet$

归约—归约冲突

G为非LR(0)

用SLR(1)方法

FOLLOW(A) = {a,c}

FOLLOW(B) = {a,c}

$\cap \neq \emptyset$

G为非SLR(1)

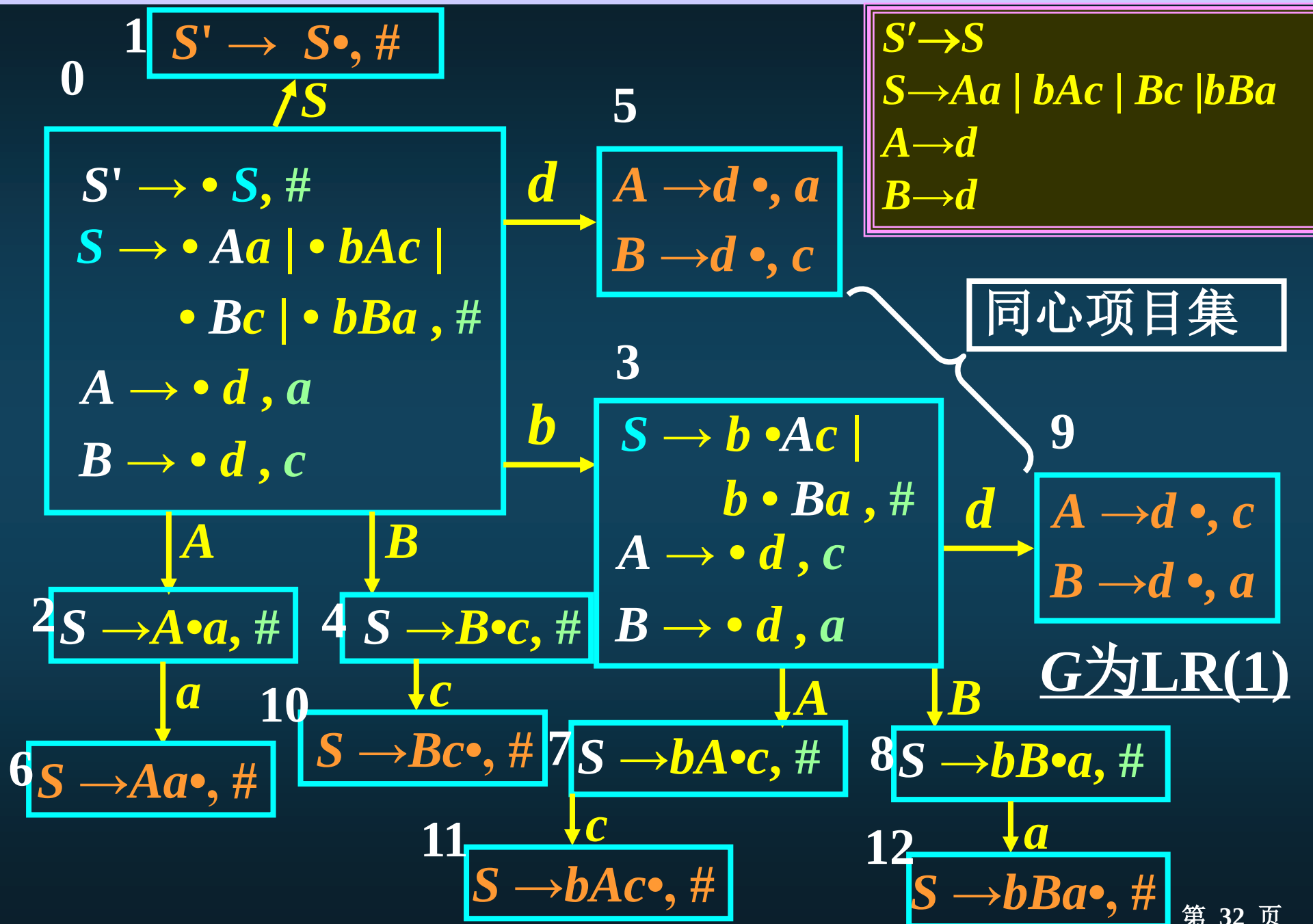
$S' \to S$

$S \to Aa \mid bAc \mid Bc \mid bBa$

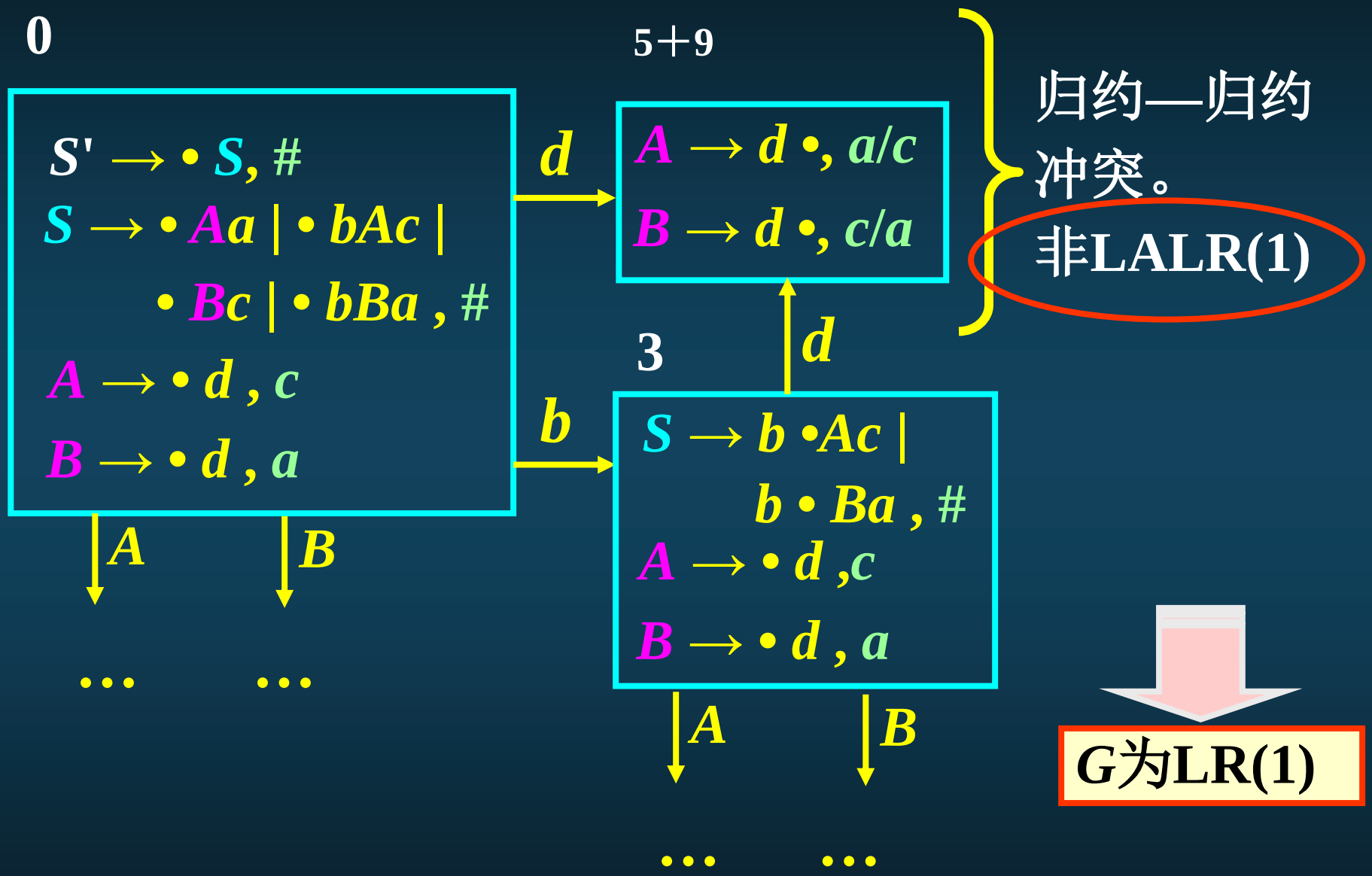
$A \to d$

$B \to d$

$S' \to S$
$S \to Aa \mid bAc \mid Bc \mid bBa$
$A \to d$
$B \to d$



G为LR(1)


0
S' → • S, #
S → • Aa | • bAc |
• Bc | • bBa , #
A → • d , c
B → • d , a
A
B
…
…
d
5+9
A → d •, a/c
B → d •, c/a
b
3
S → b •Ac |
b • Ba , #
A → • d ,c
B → • d , a
d
A
B
…
…
归约—归约
冲突。
非LALR(1)
G为LR(1)

5.1 “移近—归约”分析法

5.2 算符优先分析法

5.3 LR分析鸟瞰

5.4 LR (0)分析

5.5 SLR (1)分析

5.6 LR (1)分析

5.7 LALR (1)分析

5.8 LR分析对二义文法的应用

5.9 LR分析的错误处理与恢复

5.10 语法分析器的自动生成与YACC

- **定理5.2**

任何一个二义文法都不是一个LR文法。

** 二义文法会导致语法分析的二义性。

** 二义文法的有用之处在于可以缩小文法的规模。

例如，G：$E \rightarrow E+E \mid E*E \mid (E) \mid i$

G'：　$E \rightarrow T \mid E+T$

$T \rightarrow F \mid T*F$

$F \rightarrow (E) \mid i$

例如，　G：　$S \rightarrow iSeS \mid iS \mid a$

或　G'：$S \rightarrow \{iSeS\} \mid \{iS\} \mid a$

例5.19 设有文法$G(E)$：

$E \rightarrow E+E$

$E \rightarrow E*E$

$E \rightarrow (E)$

$E \rightarrow i$

拓广文法：

$E' \rightarrow E$

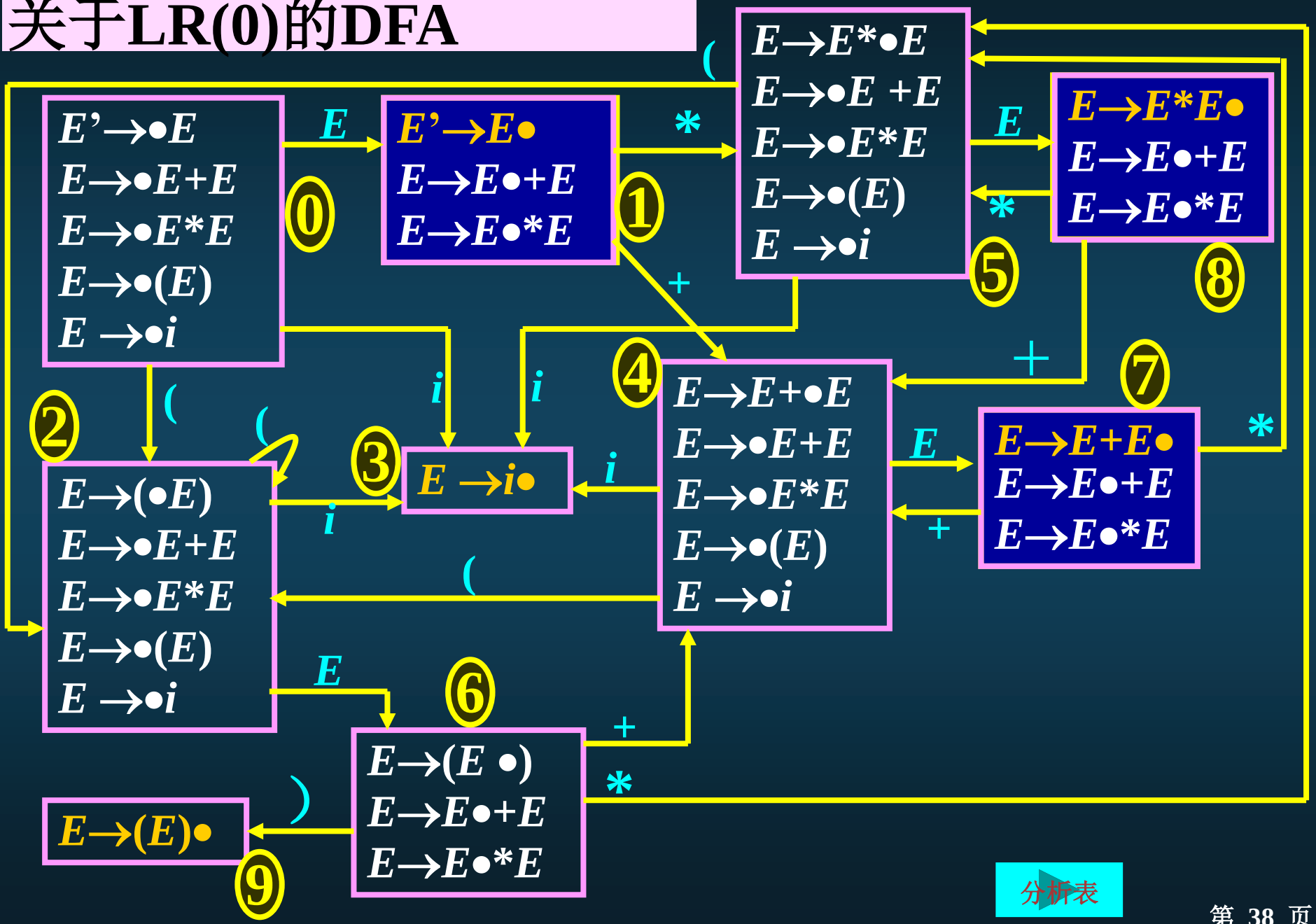
$E \rightarrow E+E$ ①

$E \rightarrow E*E$ ②

$E \rightarrow (E)$ ③

$E \rightarrow i$ ④

关于LR(0)的DFA



分析表

①

$E' \to E\bullet$

$E \to E\bullet + E$

$E \to E\bullet * E$

⑦

$E \to E+E\bullet$

$E \to E\bullet + E$

$E \to E\bullet * E$

⑧

$E \to E*E\bullet$

$E \to E\bullet + E$

$E \to E\bullet * E$

FOLLOW(E')＝

{#}∩{{＋}∪{*}}= Φ

FOLLOW(E)＝

{＋, *,), # } ∩ {{＋} ∪ {*}}

≠ Φ

$E' \to E$ $E \to E+E$ ① $E \to E*E$ ②

$E \to (E)$ ③ $E \to i$ ④

$E' \to E$　① $E \to E+E$　② $E \to E*E$　③ $E \to (E)$　④ $E \to i$

FOLLOW(E)＝ {＋, ＊,) , # }

文法G(E)的LR分析表

状 态	ACTION						GOTO
	i	+	*	(	)	#	E
0	s_3			s_2			1
1		s_4	s_5			acc	
2	s_3			s_2			6
3	r_4	r_4	r_4	r_4	r_4	r_4	
4	s_3			s_2			7
5	s_3			s_2			8
6		s_4	s_5		s_9		
7		r_1	s_5		r_1	r_1	
8		r_2	r_2		r_2	r_2	
9	r_3	r_3	r_3	r_3	r_3	r_3	

DFA

$: *i*+*i*+*i* #

F

action(7 , +) = r_2

7	*E*
4	+
1	*E*
0	#

P

E

$: *i*+*i***i* #

F

action(7 , *) = S_5

5	*
7	*E*
4	+
1	*E*
0	#

P

例5.20　设有文法G:

$S' \rightarrow S$

$S \rightarrow iSeS \mid iS \mid a$　　①|②|③

其中：

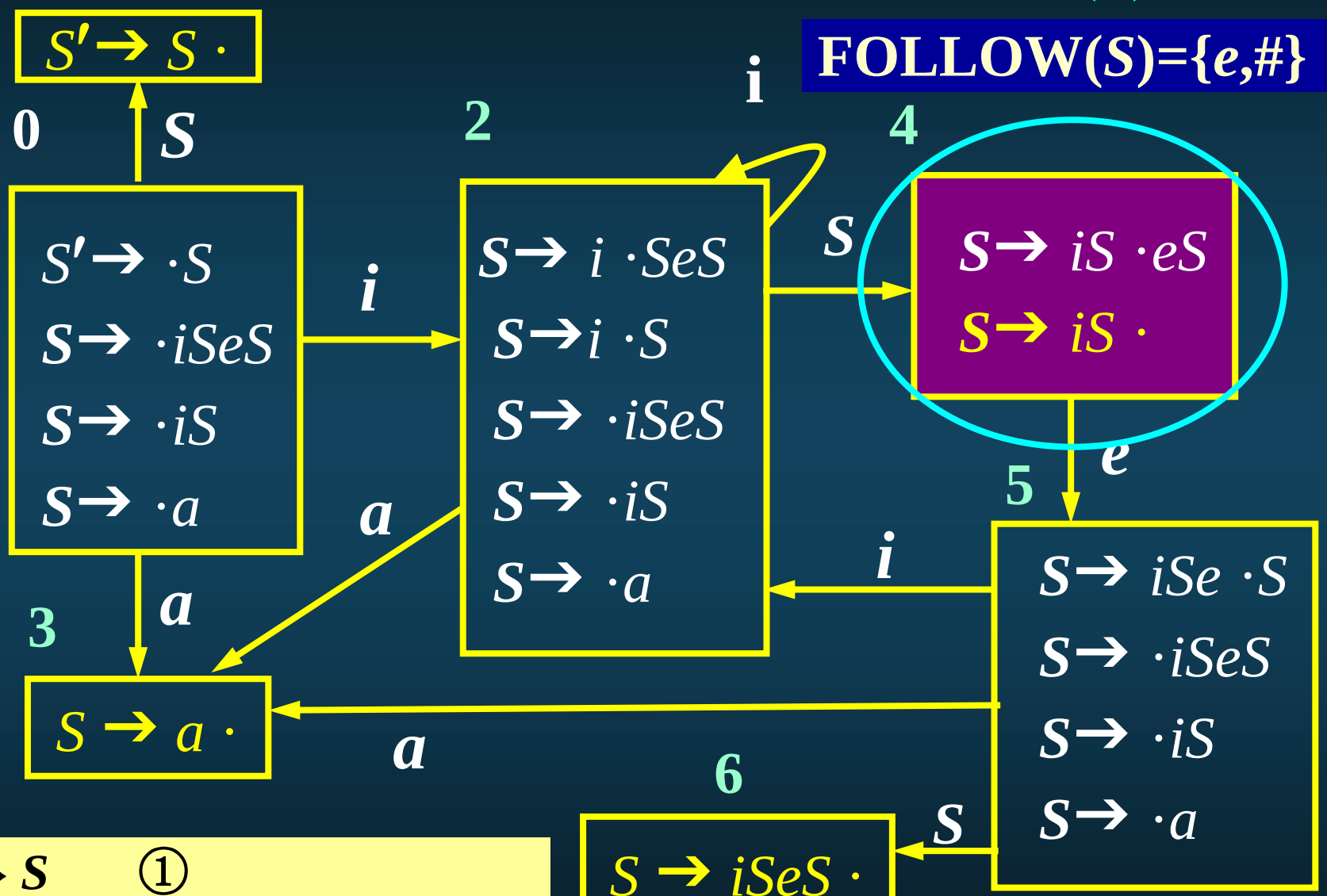
i : if e_r then

e: else

a, S: 语句

文法G的LR(0)的DFA

FOLLOW(S)={e,#}



$S' \rightarrow S$　①

$S \rightarrow iSeS \mid iS \mid a$　②|③|④

分析表

DFA

文法G的LR分析表

	i	*e*	*a*	#	*S*
0	S_2		S_3		1
1				acc	
2	S_2		S_3		4
3	r_3	r_3	r_3	r_3	
4		S_5		r_2	
5	S_2		S_3		6
6	r_1	r_1	r_1	r_1	

分析

$S' \rightarrow S$

$S \rightarrow iSeS \mid iS \mid a$ ① | ② |③

对$ = *iiaea#* 进行分析

if e_1 then	if e_2 then	S_1	else	S_2
i	*i*	*a*	*e*	*a*

$: *i i a e a #*

F

action(4 , *e*) = S_5

5	*e*
4	*S*
2	*i*
2	*i*
0	#

F++;P++

P

分析表

5.1 “移近—归约”分析法

5.2 算符优先分析法

5.3 LR分析鸟瞰

5.4 LR (0)分析

5.5 SLR (1)分析

5.6 LR (1)分析

5.7 LALR (1)分析

5.8 LR分析对二义文法的应用

5.9 LR分析的错误处理与恢复

5.10 语法分析器的自动生成与YACC

- 错误存在的必然性

问题复杂性、程序员素质、输入错、系统环境生疏 …

- 编译中的错误种类

1. 词法错误；
2. 语法错误；
3. 语义错误(静态、动态)；
4. 违反环境限制的错误；

- 错误处理基本目标

1. 清晰准确地报告错误（错误的定性和定位）；
2. 迅速从每个错误中恢复过来，以便诊 断后面的错误；

- 错误处理与恢复策略

1. 局部化恢复

2. 出错产生式

扩充语言的文法，增加产生错误结构的产生式。分析中可以直接识别处理错误。

3. 全局纠正

获取全局最小代价纠正。

- 局部化恢复的一般性技术

1. 紧急恢复方式

编译器每次发现错误时，抛弃一个输入符号，直到输入符号属于某个指定的同步符号集合为止。

例如，界限符：

；、}、end …

思想：在分析到某一含有错误的短语时，分析程序认为含有错误的短语是由某一非终结符*A*所推导出的，短语的一部分已处理，放在栈顶部一系列状态中，剩下未处理的在输入串，分析程序跳过这些剩余符号，直至找到A的跟随字符为止，同时把栈顶内容逐个移去，直至找到某个状态*q*，GOTO(*q*,*A*)对应一新状态，将*q*，*A*压入栈。

2. 短语级恢复

发现错误时，对剩余符号串作局部校正。即使用可以使编译器继续工作的输入串代替剩余输入的前缀。

在输入串的出错点采用插入、删除或修改的方法。

* 关键：选择合适的替换串。

(*i*+*i* ; ⇨ (*i*+*i*) ;

*i i***i*; ⇨ *i* + *i***i*;

- 错误处理基本目标

1. 清晰准确地报告错误（错误的定性和定位）；
2. 迅速从每个错误中恢复过来，以便诊 断后面的错误；

- 错误恢复的困难

1. 源程序书写的灵活性；
2. 错误发生的随机性；
3. 不使正确程序的处理效率降低。
4. 可能校正途径的多样性。

例5.19　设有文法$G(E)$：

$E' \rightarrow E$

$E \rightarrow E+E$　　①

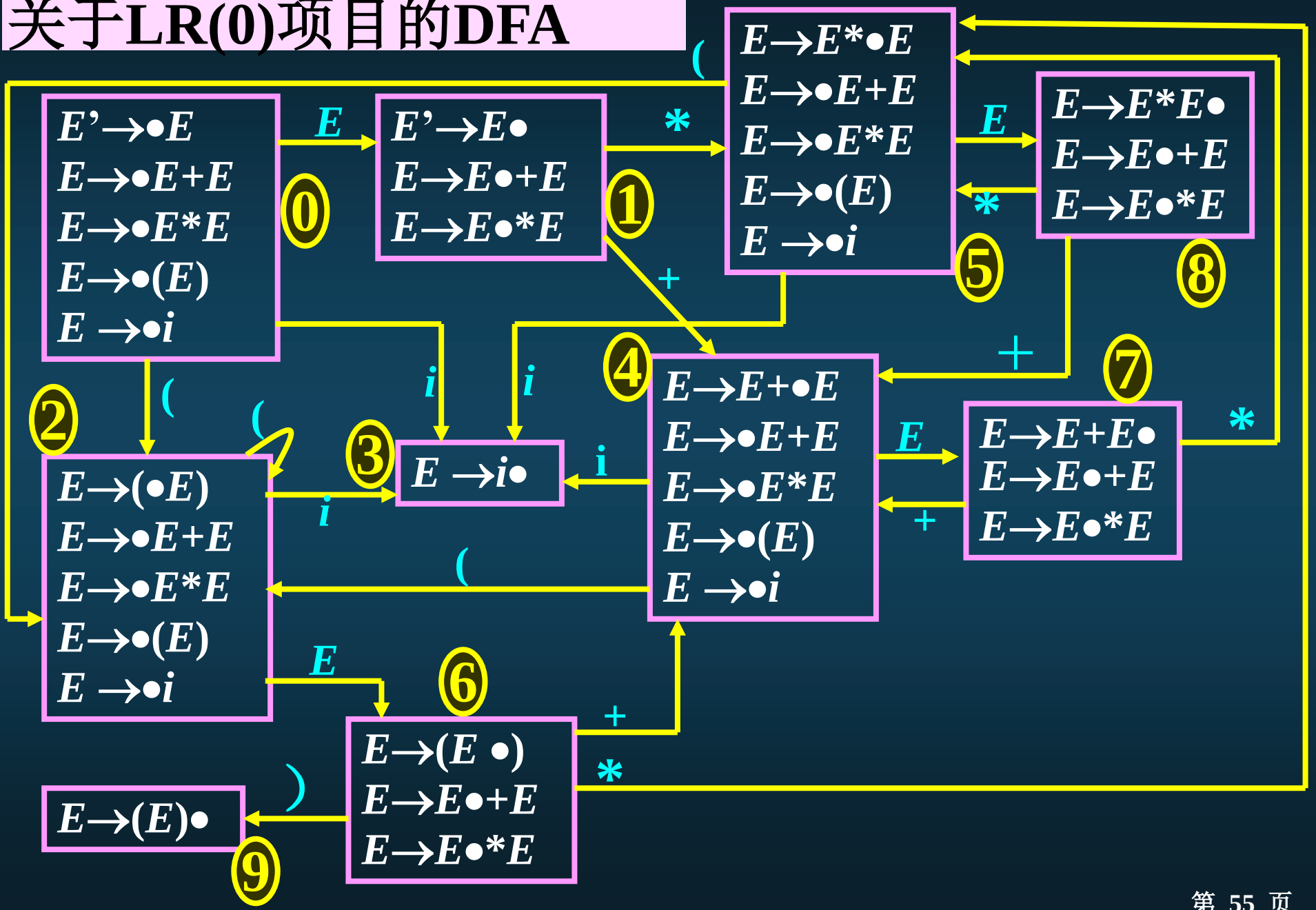
$E \rightarrow E*E$　　②

$E \rightarrow (E)$　　③

$E \rightarrow i$　　④

构造文法$G(E)$的带出错处理的LR分析表

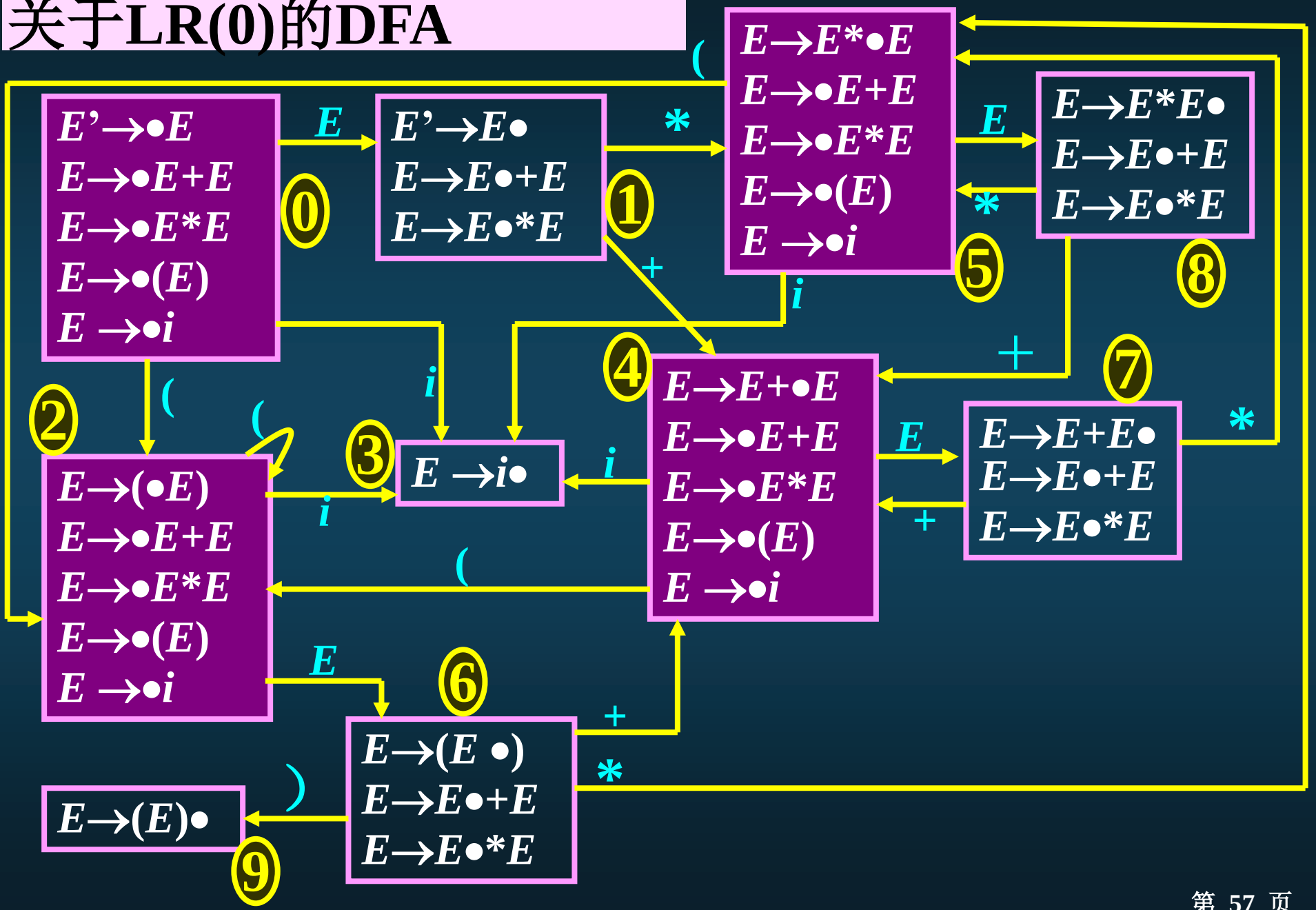
关于LR(0)项目的DFA



文法G(E)的LR分析表

状 态	ACTION						GOTO
	i	+	*	(	)	#	*E*
0	s_3			s_2			1
1		s_4	s_5			acc	
2	s_3			s_2			6
3	r_4	r_4	r_4	r_4	r_4	r_4	
4	s_3			s_2			7
5	s_3			s_2			8
6		s_4	s_5		s_9		
7	r_1	r_1	s_5	r_1	r_1	r_1	
8	r_2	r_2	r_2	r_2	r_2	r_2	
9	r_3	r_3	r_3	r_3	r_3	r_3	

关于LR(0)的DFA


E'→•E
E→•E+E
E→•E*E
E→•(E)
E →•i
0
E
E'→E•
E→E•+E
E→E•*E
1
*
E→E*•E
E→•E+E
E→•E*E
E→•(E)
E →•i
5
E
E→E*E•
E→E•+E
E→E•*E
8
*
+
i
4
E→E+•E
E→•E+E
E→•E*E
E→•(E)
E →•i
7
E
E→E+E•
E→E•+E
E→E•*E
*
+
i
(
2
E→(•E)
E→•E+E
E→•E*E
E→•(E)
E →•i
(
3
E →i•
i
i
(
E
6
E→(E •)
E→E•+E
E→E•*E
+
*
)
E→(E)•
9

e1:

处于状态0、2、4、5时，要求输入符号为运算对象 *i* 或（*E*），此时若遇到+、*、#，则调用e1。

e1功能：

将假设的 *i* 和状态3入栈；(shift)

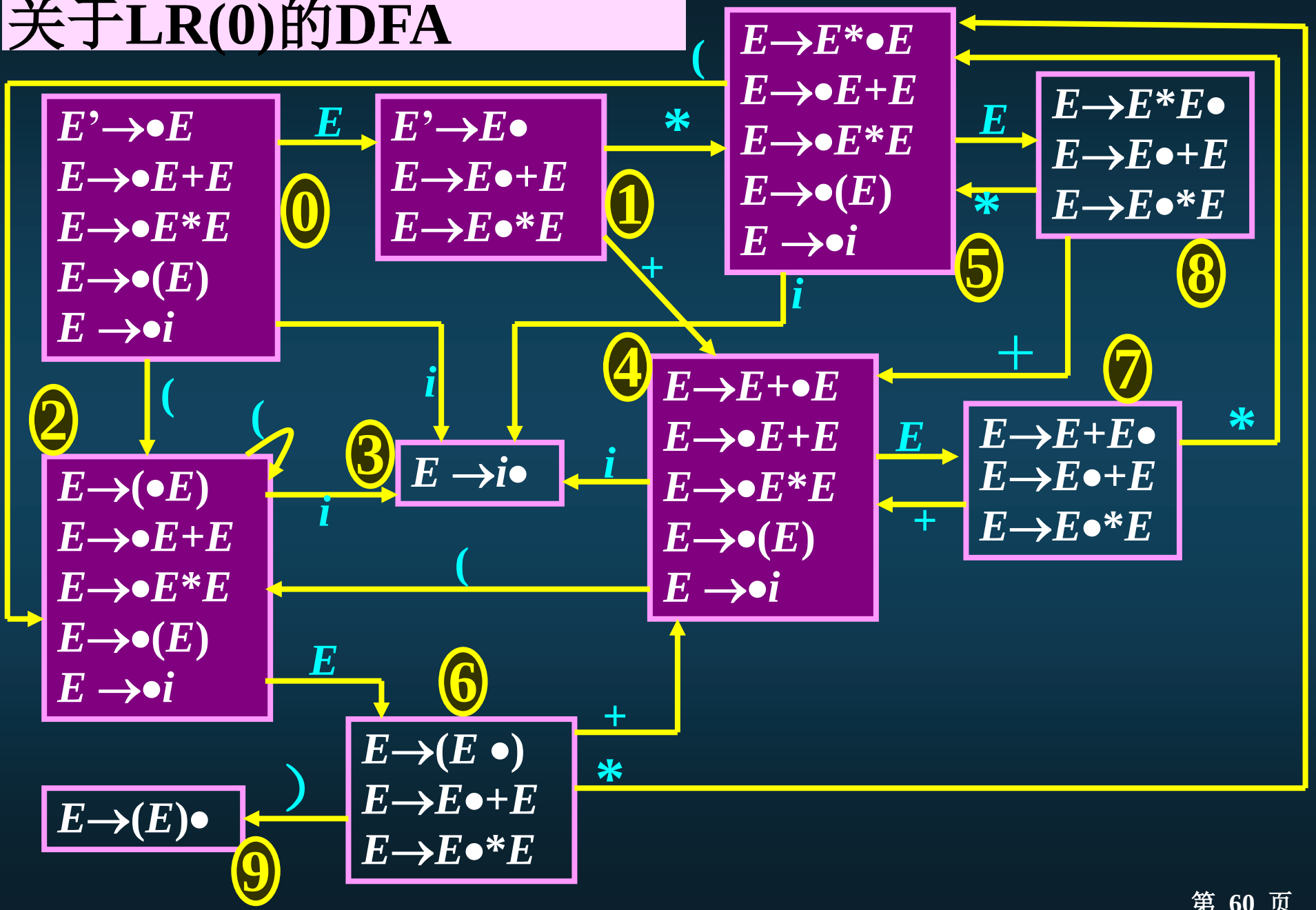
e1出错信息：

缺少运算对象。

文法G(E)的LR分析表

状 态	ACTION						GOTO
	i	+	*	(	)	#	E
0	s_3	e1	e1	s_2		e1	1
1		s_4	s_5			acc	
2	s_3	e1	e1	s_2		e1	6
3	r_4	r_4	r_4	r_4	r_4	r_4	
4	s_3	e1	e1	s_2		e1	7
5	s_3	e1	e1	s_2		e1	8
6		s_4	s_5		s_9		
7	r_1	r_1	s_5	r_1	r_1	r_1	
8	r_2	r_2	r_2	r_2	r_2	r_2	
9	r_3	r_3	r_3	r_3	r_3	r_3	

关于LR(0)的DFA


E'→•E
E→•E+E
E→•E*E
E→•(E)
E →•i
0
E
E'→E•
E→E•+E
E→E•*E
1
*
E→E*•E
E→•E+E
E→•E*E
E→•(E)
E →•i
5
(
E
E→E*E•
E→E•+E
E→E•*E
8
*
+
i
+
4
E→E+•E
E→•E+E
E→•E*E
E→•(E)
E →•i
7
E
E→E+E•
E→E•+E
E→E•*E
*
+
i
i
E →i•
3
2
(
(
E→(•E)
E→•E+E
E→•E*E
E→•(E)
E →•i
i
(
E
6
E→(E •)
E→E•+E
E→E•*E
+
*
)
E→(E)•
9

e2:

处于状态0、1、2、4、5时，若遇到“）”，则

调用e2。

e2功能：

删除输入的“）”；　(F++)

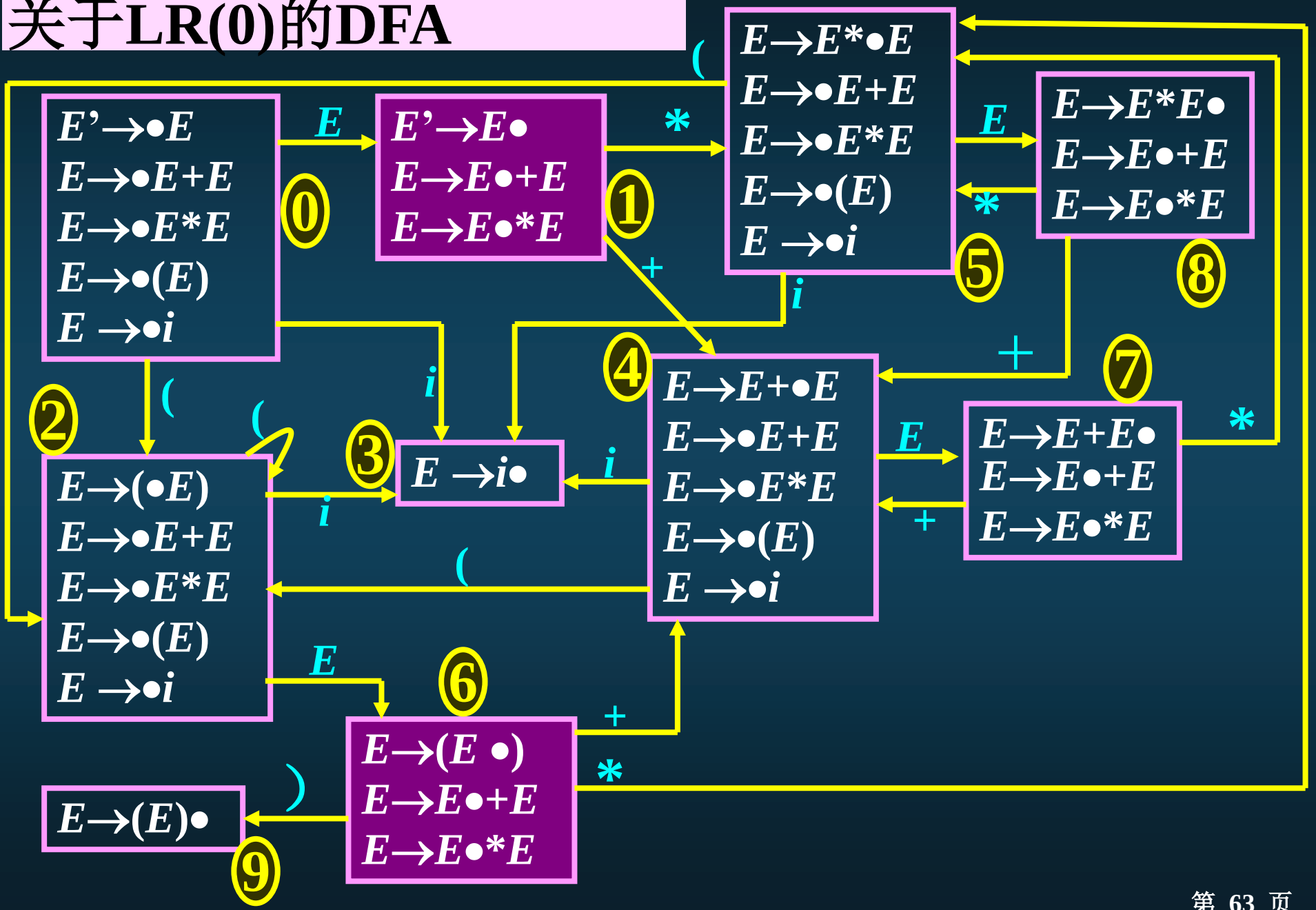
e2出错信息：

“ ）”不配对。

文法G(E)的LR分析表

状 态	ACTION						GOTO
	i	+	*	(	)	#	E
0	s_3	e1	e1	s_2	e2	e1	1
1		s_4	s_5		e2	acc	
2	s_3	e1	e1	s_2	e2	e1	6
3	r_4	r_4	r_4	r_4	r_4	r_4	
4	s_3	e1	e1	s_2	e2	e1	7
5	s_3	e1	e1	s_2	e2	e1	8
6		s_4	s_5		s_9		
7	r_1	r_1	s_5	r_1	r_1	r_1	
8	r_2	r_2	r_2	r_2	r_2	r_2	
9	r_3	r_3	r_3	r_3	r_3	r_3	

关于LR(0)的DFA


E'→•E
E→•E+E
E→•E*E
E→•(E)
E →•i
0
E
E'→E•
E→E•+E
E→E•*E
1
*
E→E*•E
E→•E+E
E→•E*E
E→•(E)
E →•i
5
(
E
E→E*E•
E→E•+E
E→E•*E
8
*
+
i
+
4
E→E+•E
E→•E+E
E→•E*E
E→•(E)
E →•i
7
E
E→E+E•
E→E•+E
E→E•*E
*
+
i
i
E →i•
3
2
(
(
E→(•E)
E→•E+E
E→•E*E
E→•(E)
E →•i
i
(
E
6
E→(E •)
E→E•+E
E→E•*E
+
*
)
E→(E)•
9

e3:

处于状态1或6时，期望下面输入符为运算符或“)”，但遇到“*i*”或“(”时，则调用e3。

e3功能：

将假设的“+”和状态 4入栈, (P++)；

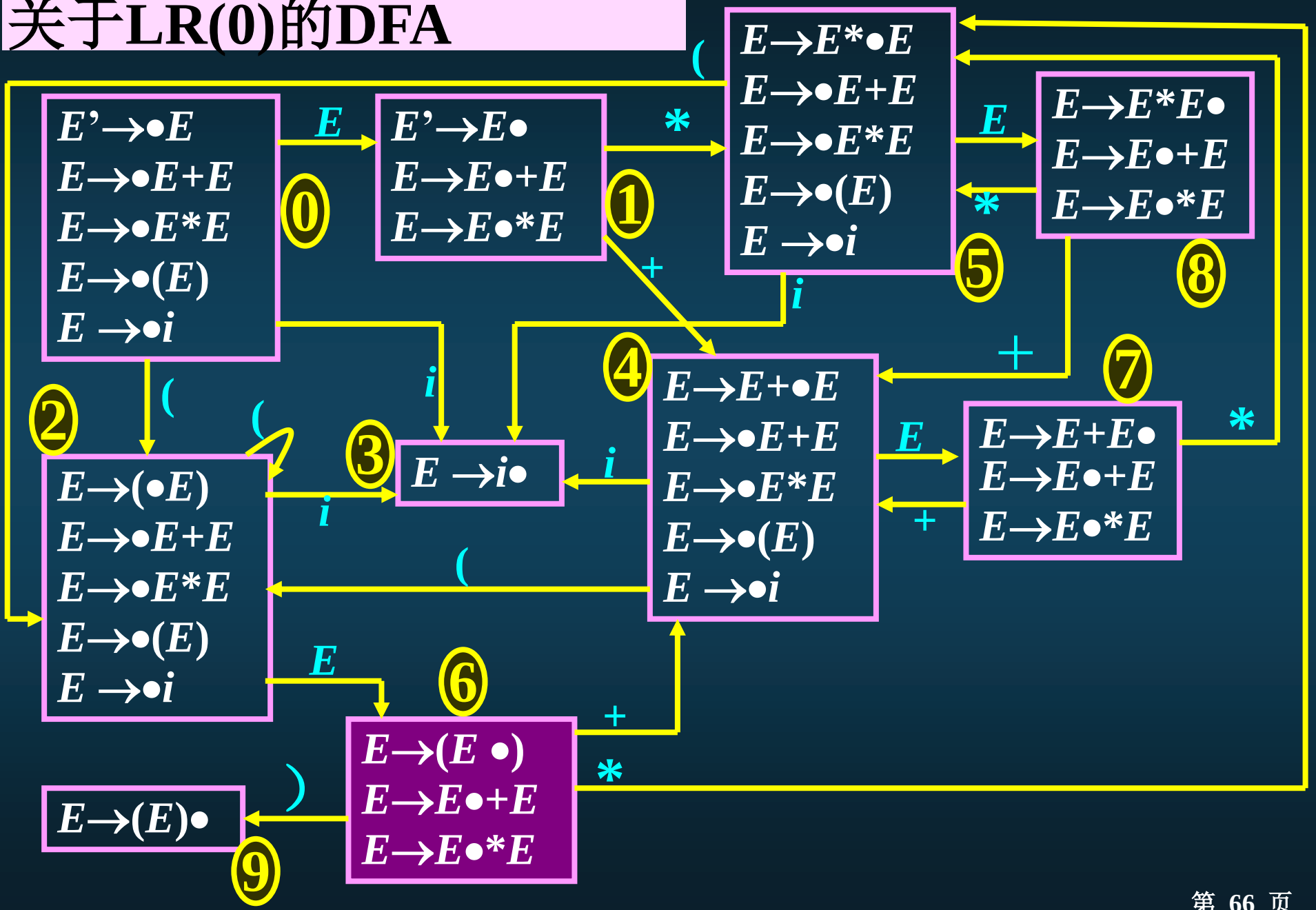
e3出错信息：

缺少运算符。

文法G(E)的LR分析表

状 态	ACTION						GOTO
	i	+	*	(	)	#	E
0	s_3	e1	e1	s_2	e2	e1	1
1	e3	s_4	s_5	e3	e2	acc	
2	s_3	e1	e1	s_2	e2	e1	6
3	r_4	r_4	r_4	r_4	r_4	r_4	
4	s_3	e1	e1	s_2	e2	e1	7
5	s_3	e1	e1	s_2	e2	e1	8
6	e3	s_4	s_5	e3	s_9		
7	r_1	r_1	s_5	r_1	r_1	r_1	
8	r_2	r_2	r_2	r_2	r_2	r_2	
9	r_3	r_3	r_3	r_3	r_3	r_3	

关于LR(0)的DFA


E'→•E
E→•E+E
E→•E*E
E→•(E)
E →•i
0
E
E'→E•
E→E•+E
E→E•*E
1
*
(
E→E*•E
E→•E+E
E→•E*E
E→•(E)
E →•i
5
E
E→E*E•
E→E•+E
E→E•*E
8
*
+
i
+
4
E→E+•E
E→•E+E
E→•E*E
E→•(E)
E →•i
7
E
E→E+E•
E→E•+E
E→E•*E
*
+
i
i
E →i•
3
2
(
(
E→(•E)
E→•E+E
E→•E*E
E→•(E)
E →•i
i
(
E
6
E→(E •)
E→E•+E
E→E•*E
+
*
)
E→(E)•
9

e4:

处于状态6时，期望下面输入符为运算符或 “）”，但遇到“#” 时，则调用e4。

e4功能：

将假设的“）”和状态9入栈, (P++) ;

e4出错信息：

缺少“ ）”。

文法G(E)的LR分析表(带出错处理)

举例2 举例1

状 态	ACTION						GOTO
	i	+	*	(	)	#	E
0	s_3	e1	e1	s_2	e2	e1	1
1	e3	s_4	s_5	e3	e2	acc	
2	s_3	e1	e1	s_2	e2	e1	6
3	r_4	r_4	r_4	r_4	r_4	r_4	
4	s_3	e1	e1	s_2	e2	e1	7
5	s_3	e1	e1	s_2	e2	e1	8
6	e3	s_4	s_5	e3	s_9	e4	
7	r_1	r_1	s_5	r_1	r_1	r_1	
8	r_2	r_2	r_2	r_2	r_2	r_2	
9	r_3	r_3	r_3	r_3	r_3	r_3	

e1: 将假设的 *i* 和状态3入栈；(shift)；出错信息： 缺少运算对象。

e2:删除输入的“）”； (F++)；出错信息：“ ）”不配对。

e3: 将假设的 “+” 和状态 4入栈, (P++)；出错信息：缺少运算符。

e4: 将假设的“）”和状态9入栈, (P++)；出错信息： 缺少“ ）”。

$1: (*i*+*i* # ⇒ $: (*i*+*i*) # ⇒ action(9 , #) = r_3

F

7	*E*
4	+
6	*E*
2	(
0	#

E

P

F

9	)
6	*E*
2	(
0	#

E

P

P

goto(2, *E*) =6

action(6 , #) = e4

action(7, #) = r_1

$2: *i i* # → $: *i*+*i* # → action(4 , *i*) = S_3

F

3	*i*
4	+
1	*E*
0	#

P

action(1 , *i*) = e3

5.1 “移近—归约”分析法
5.2 算符优先分析法
5.3 LR分析鸟瞰
5.4 LR (0)分析
5.5 SLR (1)分析
5.6 LR (1)分析
5.7 LALR (1)分析
5.8 LR分析对二义文法的应用
5.9 LR分析的错误处理与恢复
5.10 语法分析器的自动生成与YACC

5.10.1 YACC简介

5.10.2 YACC语言

5.10.3 YACC处理二义文法

YACC（Yet Another Compiler-Compiler）

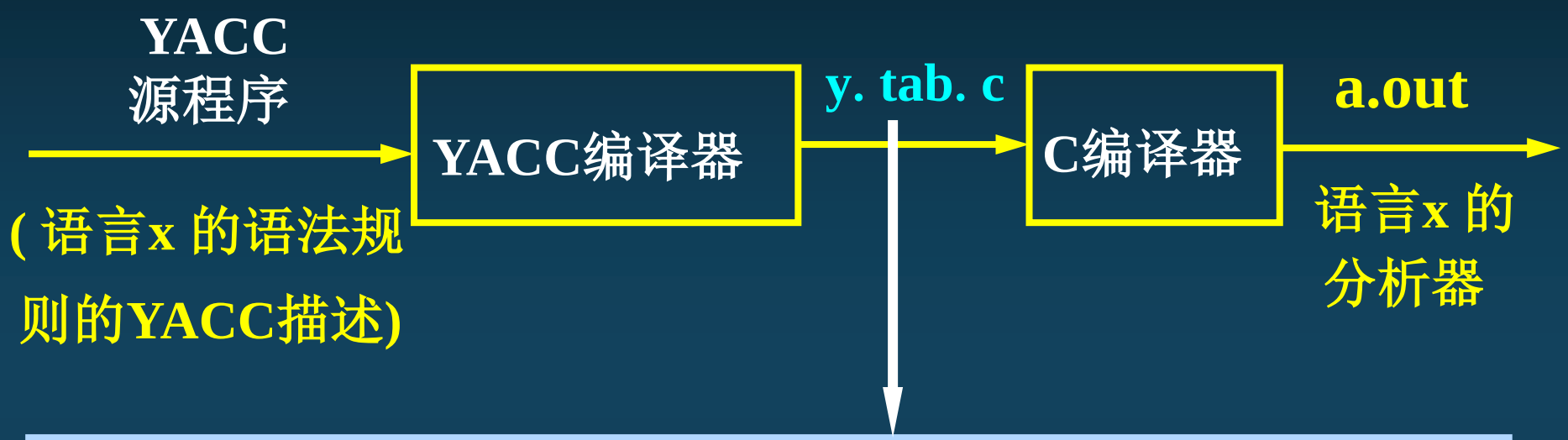
parser generator

Bison（GNU工程推出，与YACC向上兼容。）

1. 是一个LALR(1)分析器自动生成器；
2. 与LEX有标准接口；
3. 适应二义文法的LALR(1)分析；
4. 可带语义处理。

- **Compiler — Compiler的功能：**

YACC自动构造分析器的模式

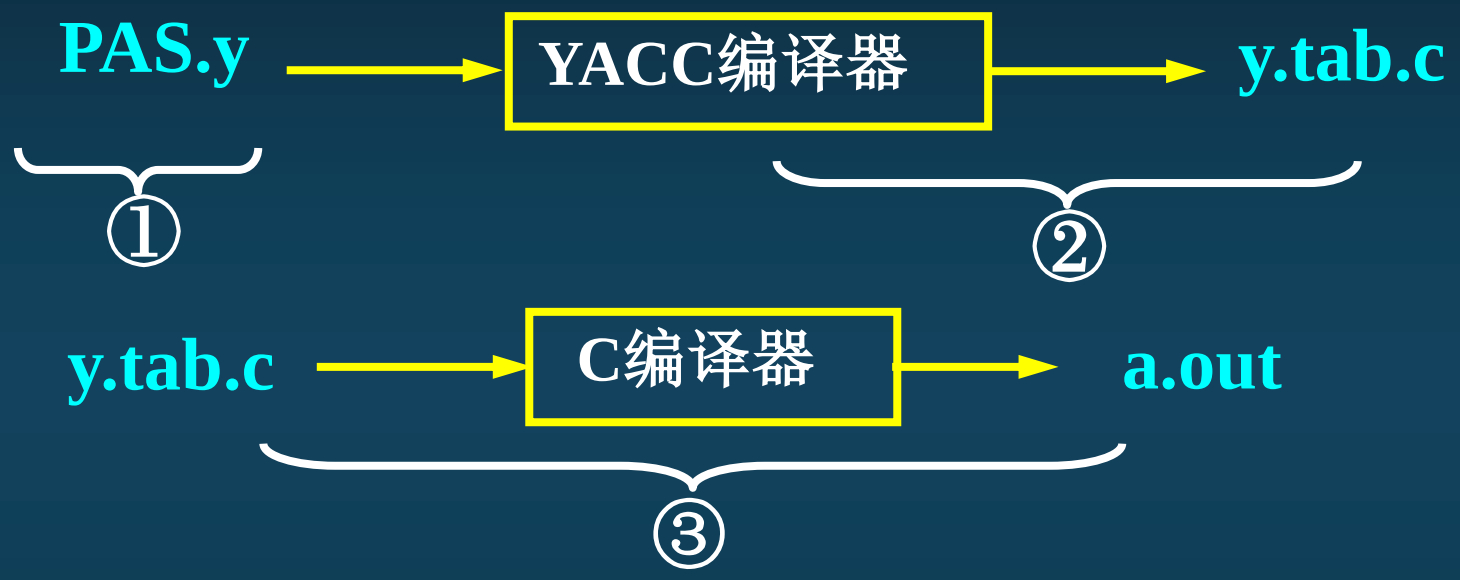


UNIX命令 （编译y.tab.c）

cc y.tab.c –ly

其中：ly表示使用LR分析器的库（对分析表进行压缩存储）。

- 使用YACC步骤

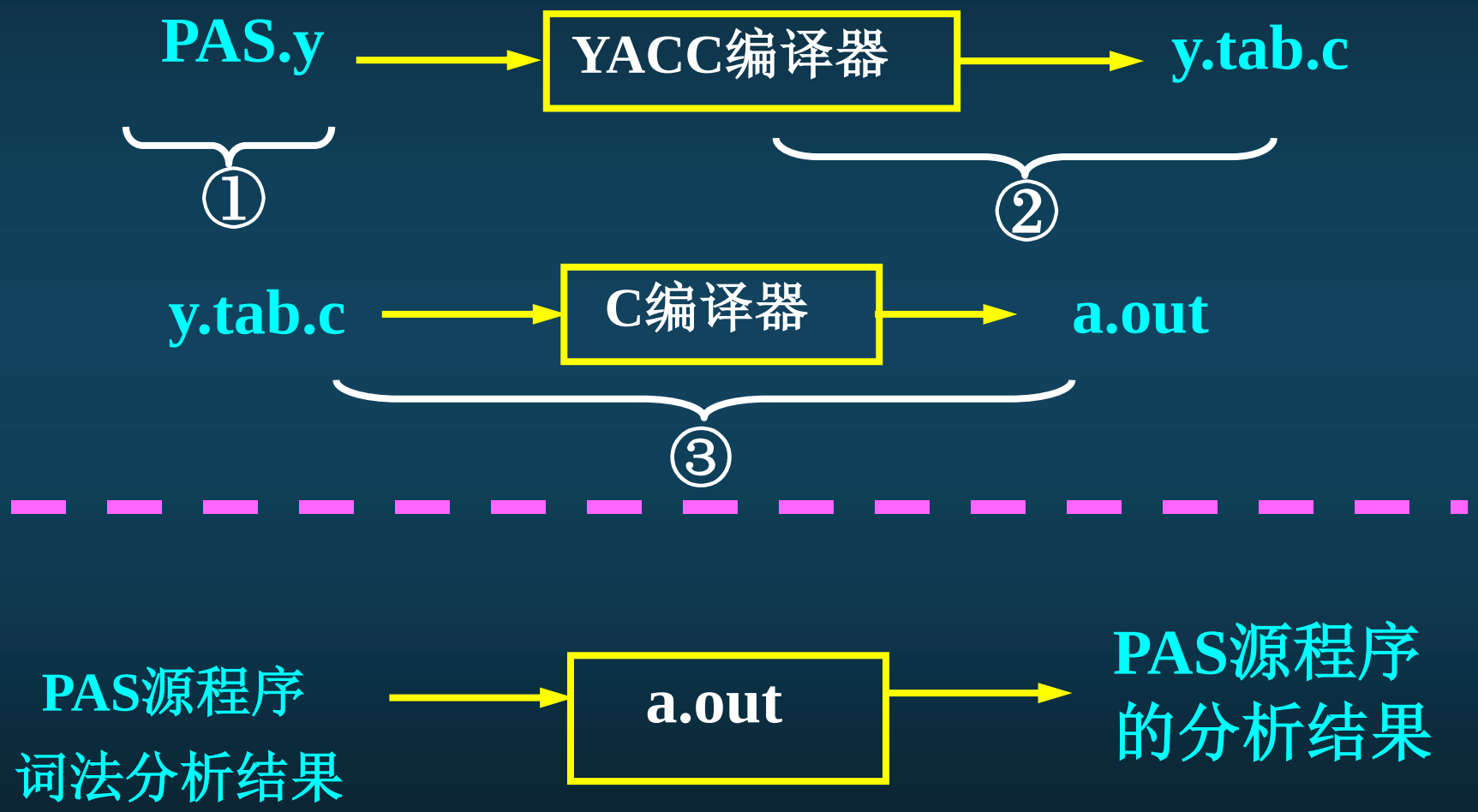


① 编辑YACC源程序 (例如，生成文本格式的PAS语言的YACC源文件PAS.y)；

② 使用命令 yacc PAS.y 运行YACC ，正确则输出 y.tab.c；

③ 调用C编译器编译 cc y.tab.c，并与其它C模块连接产生执行文件；调试执行文件，直至获得正确输出。

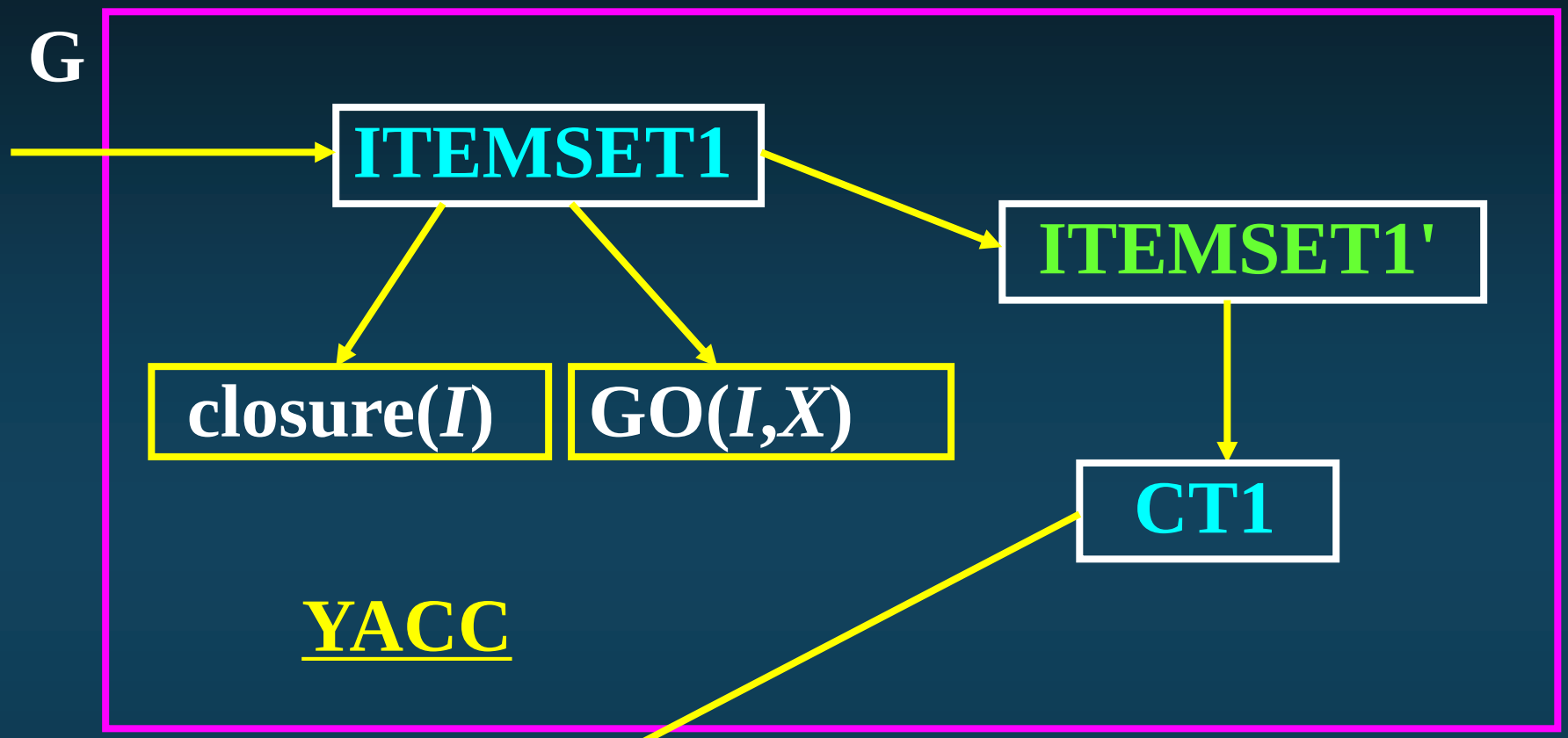
- 使用YACC步骤


PAS.y
YACC编译器
y.tab.c
①
②
y.tab.c
C编译器
a.out
③
PAS源程序
词法分析结果
a.out
PAS源程序
的分析结果

YACC体系 { YACC语言 / YACC 编译器 }

YACC实现

1. 构造LR(1)的*C* (ITEMSET1 — 算法5.9)；
2. 查找、合并同心项目集，构造LALR(1)的*C* （ITEMSET1'）；
3. 构造LALR(1)的分析表 (CT1— 算法5.11)；



- LALR(1)分析表
- LR总控程序
- Action(q_i,a_i)=r_j (+语义处理子程序)

5.10.1 YACC简介

5.10.2 YACC语言

5.10.3 YACC处理二义文法

- **YACC源程序结构**

说明部分

%%

翻译规则

%%

辅助过程

辅助子程序部分，由C语言函数构成的，如词法分析程序及错误诊断程序

- 说明部分
  - ％｛ ..... ％｝ C语言程序的常规说明(头文件 / 宏定义 ...)
  - 文法符号(一般为终结符)和文法规则的说明

    (对文法规则说明的一些限定规则和条件的声明)

    ％ 说明内容1

    ％ 说明内容2

    .....

常规说明

```
%{
        # include <ctype.h>
        # include <stdio.h>
        # define YYSTYPE double    /* YACC栈定义
                                    为double类型 */
%}
  %   token NUMBER
  %   left  '+' '-'
  %   left  '*' '/'
  %   right UMINUS
```

文法符号和文法规则的说明

- 翻译规则部分

```
%%    翻译规则1
      翻译规则2
      …
      翻译规则n
```

翻译规则 *i* : 文法产生式 { 语义动作 }

BNF

＜左部文法符号＞

→ ＜候选式1＞ | ＜候选式2＞ | … | ＜候选式 n ＞

用YACC描述的一般形式为：

＜左部文法符号＞：＜候选式1＞	{语义动作1}
| ＜候选式2＞	{语义动作2}
……	
| ＜候选式 n ＞	{语义动作 n }
;	

例5.21 设文法G(E)为 A → E+E | E*E | number

文法G(A)的YACC源程序如下：

```
%{
    #include <ctype.h>
    #include <stdio.h>
%}
 %  token number
%%
    lines: expr '\n'                { printf("%g\n", $1)};
    ;
    expr : expr '+' expr            { $$ = $1+$3; }
         | expr '*' expr            { $$ = $1*$3; }
         | number
    ;
%%
```

(辅助过程)

5.10.1 YACC简介

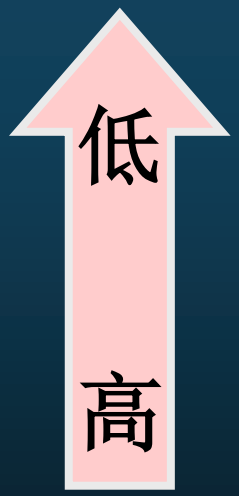
5.10.2 YACC语言

5.10.3 YACC处理二义文法

例5.22 设文法*G*(*E*)为

$$E \rightarrow E+E \mid E-E \mid E*E \mid E/E \mid (E) \mid -E \mid \text{NUMBER}$$

```
%{
        # include <ctype.h>
        # include <stdio.h>
        %}
%   token NUMBER
%   left     '+' '-'
%   left     '*' '/'
%   right    UMINUS
```



```
%%
        lines: lines expr '\n'      { printf("%g\n", $2);
                | lines '\n'
                | /* ε */
                ;
        expr: expr '+' expr             {$$=$1+$3;}
                | expr '-' expr         {$$=$1-$3;}
                | expr '*' expr         {$$=$1*$3;}
                | expr '/' expr         {$$=$1/$3;}
                | '(' expr      ')'     {$$=$2;}
                | '- ' expr     % prec UMINUS  {$$=-$2;}
                | NUMBER
                ;
```

LR分析总结

- LR(k)分析实现的核心思想：

归约条件 (句柄最左性)

⇓

句柄、活前缀关系

⇓

LR的*C* + GO(分析动作) =识别活前缀的DFA

⇓

LR分析表

- LR(0) 、SLR(1)、LR(1)、LALR(1)分析的关系与区别，相应LR文法分析器的构造。

end

0：{S’→•S,#；S→•L＝R,#；S→•R,#；L→•*R,＝/#;
L →•I,＝/#,R →•L, ＝/#}

1：{S’→S•，# }

2：{S→L•＝R，#；R→ L•，#}

3：{S→R•，#}

4+11：{L→*•R,#/=；R→•L,#/=；L→•*R,#/=;
L →•I,#/=}

5+12：{L →i•，＝/#}

6：{S→L＝ • R,#；R→•L,#；L→•*R,#；L →•i,#}

7＋13：{L→*R •,＝/#}

8+10：{R→L•,＝/#}

9：{S→L＝ R • ，# }

文法G(S')的GO函数

state	=	*	i	L	R	S
0		4	5	2	3	1
2	6					
4		4	5	8	7	
6		4+11	5+12	8+10	9	
11		11	12	10	13	

文法G(S')的合并同心项目集后GO函数

state	=	*	i	L	R	S
0		4+11	5+12	2	3	1
2	6					
4＋11		4+11	5+12	8+10	7+13	
6		4+11	5+12	8+10	9	

LR(1)

分析表